

Evaluating customer satisfaction and store revenue using agent-based modeling of grocery store layouts

Livia Tengelin
Sebastian Zulj

Systemutvecklare
Kandidatuppsats
15 hp
VT2025
Handledare: Emil Johansson
Examinator: Erdal Akin

Abstract

The design of a grocery store's interior can influence factors such as customer density, flow, waiting times, and the number of impulse purchases. These factors may in turn affect the customer's shopping experience, which can influence customer satisfaction and ultimately the store's revenues. However, knowledge about how interior design affects customer satisfaction in a shopping context remains limited. This simulation study aims to use agent-based modeling to analyze how the placement of various departments within a grocery store may affect customer satisfaction and the store's sales revenues. Using the ODD protocol, an agent-based model was designed and implemented in the NetLogo framework. The developed model was then used to run simulation experiments testing various store layouts. These simulations found the model adequate at fulfilling the goal of providing results to be used in a comparison between different layouts based on customer satisfaction and store revenue. The results suggest the usefulness of the model, both in future studies and on an industrial level.

Keywords: Agent-based modeling, customer satisfaction, grocery store layout, ODD protocol, simulation

Acknowledgment

The authors would like to thank Emil Johansson for his guidance.

Table of contents

1 Introduction **8**
- 1.1 Background . . . 8
- 1.2 Purpose and Objectives . . . 9
  - 1.2.1 Research Gap . . . 9
  - 1.2.2 Research Question . . . 9
- 1.3 Use of Generative AI . . . 9
- 1.4 Structure of the Text . . . 10

2 Literature Review **11**
- 2.1 Agent-Based Modeling . . . 11
- 2.2 The Layout Problem . . . 12
- 2.3 Customer Satisfaction . . . 12
- 2.4 Summary and Research Gap . . . 13

3 Methodology **14**
- 3.1 Design Science Research Methodology . . . 14
- 3.2 Problem Identification & Motivation . . . 15
  - 3.2.1 Identifying sources . . . 15
  - 3.2.2 Evaluation of the Identified Sources . . . 16
  - 3.2.3 Review and Compilation of Sources . . . 16
- 3.3 Definition of Objectives . . . 16
- 3.4 Design and development . . . 17
  - 3.4.1 The ODD protocol . . . 17
  - 3.4.2 NetLogo . . . 17
  - 3.4.3 Previous research as a framework . . . 17
  - 3.4.4 The model . . . 17
- 3.5 Evaluation . . . 21

4 Results **22**
- 4.1 Source Code . . . 22
- 4.2 Layouts . . . 22
  - 4.2.1 Layout 1 . . . 22
  - 4.2.2 Layout 2 . . . 23
  - 4.2.3 Layout 3 . . . 24
  - 4.2.4 Layout 4 . . . 25

4.3 Simulations . . . 26
4.3.1 Structure . . . 26
4.3.2 Results . . . 26
4.4 Model Validation . . . 26

5 Discussion **28**
5.1 Simulation Results . . . 28
5.2 The Model as a Result . . . 29
5.2.1 Use of the Model . . . 29
5.3 Limitations and Weaknesses . . . 30
5.3.1 Big Data . . . 30
5.3.2 The Complexity of Customer Satisfaction . . . 30
5.3.3 Model Incompleteness . . . 31
5.3.4 Previous Research as a Framework . . . 31

6 Conclusion **32**
6.1 Summary . . . 32
6.2 Implications . . . 32
6.3 Future Work . . . 32

A ODD Description of the Model **39**
A.1 Purpose and patterns . . . 39
A.1.1 Purpose . . . 39
A.1.2 Patterns . . . 39
A.2 Entities, state variables and scales . . . 40
A.2.1 Entities and state variables . . . 40
A.2.2 Scales . . . 42
A.3 Process overview and scheduling . . . 42
A.3.1 Initialization . . . 42
A.3.2 Execution . . . 43
A.4 Design concepts . . . 46
A.4.1 Basic principles . . . 46
A.4.2 Emergence . . . 47
A.4.3 Adaptation . . . 47
A.4.4 Objectives . . . 47
A.4.5 Learning . . . 47
A.4.6 Prediction . . . 47
A.4.7 Sensing . . . 47
A.4.8 Interaction . . . 47
A.4.9 Stochasticity . . . 48
A.4.10 Collectives . . . 48
A.4.11 Observation . . . 48
A.5 Initialization . . . 48
A.5.1 Items . . . 48
A.5.2 Sections . . . 49
A.5.3 POS (Points of Sale) . . . 49
A.5.4 Doors . . . 50

A.5.5 Patches 50
A.6 Input data 50
A.7 Submodels 50
A.7.1 check-for-items-to-shop 51
A.7.2 customer-check-out 51
A.7.3 detect-closeby-customers 51
A.7.4 exit-store-attempt 51
A.7.5 get-in-line-attempt 53
A.7.6 move 53
A.7.7 pick-up-item-attempt 53
A.7.8 POS-go 53
A.7.9 queue-forward-step 54
A.7.10 randomize-shopping-list 54
A.7.11 remove-item-from-section 55
A.7.12 remove-some-items 55
A.7.13 set-target 55
A.7.14 set-up-random-customer 55
A.7.15 set-xy-pos-from-variables 57
A.7.16 set-xy-variables 57
A.7.17 spawn-new-customer 57
A.7.18 spawn-new-item? 57
A.7.19 Print Procedures 58

Figures

3.1 Overview diagram of the process 14
3.2 Entity relationship diagram of agents in the model 18
3.3 Activity diagram: The customer's life cycle 19
3.4 Activity diagram: One time unit for a customer 20

4.1 Layout 1 23
4.2 Layout 2 24
4.3 Layout 3 24
4.4 Layout 4 25

A.1 Activity diagram: *go* 43
A.2 Procedure calls from *customer-go* 45
A.3 Procedure calls from *POS-go* 46
A.4 Procedure calls from *spawn-new-customer?* 46
A.5 Activity diagram: detect-closeby-customers 52
A.6 Activity diagram: *POS-go* 54
A.7 Activity diagram: *set-target* 56
A.8 Activity diagram: *spawn-new-item?* 58

Tables

4.1 Color and department scheme for layout 1 23
4.2 Average simulation results of 50 runs of 5000 *ticks* for the 4 layouts 26

A.1 Customer entity state variables 40
A.2 Section entity state variables 41
A.3 Item entity state variables 41
A.4 Checkout entity state variables 41
A.5 Door entity state variables 41
A.6 Initialization of a customer 56
A.7 Input and output for *set-xy-variables* 57

Glossary

Agent	An autonomous actor in a system. Agents may interact with other agents or with the environment.
Agent based modeling	Modeling paradigm in which a system is modeled from the perspective of the autonomous agents that the system consists of.
GUI (Graphical User Interface)	The graphical component of a program with which the user interacts.
ODD protocol	Overview, design concept and details. This protocol is used to conceptually define agent based models in order to simplify the implementation process of the model.
Patch	A stationary square in NetLogo's grid-based world. Patches represent the environment and can store values such as color or other variables that affect agent behavior.
Tick	The unit of time in a NetLogo simulation. One tick typically represents one full update cycle in which all agents act and the environment may change.

Kapitel 1

Introduction

1.1 Background

Agent-based modeling is a research area that has, in recent years, found increasing applications across a range of domains such as behavioral science, computer science, epidemiology, and more. One of the earliest applications involved studying how individual choices and preferences can lead to segregation [1]. Furthermore, agent-based models have been developed to understand and manage the spread of viral diseases such as COVID-19 [2] and HIV [3]. One of the main advantages of agent-based models and simulations is their ability to provide insights into how individual actors (agents) in a complex system interact with each other or with their environment, and how macrostructures can emerge from these interactions [4].

Several studies have also explored the applicability of agent-based models in the retail grocery sector. For example, attempts have been made to simulate consumer behavior and purchasing habits at a regional level [5, 6, 7], where agent-based modeling has shown potential to support planning efforts in the grocery trade and to capture consumer behavior and habits [6, 7], as well as to predict store revenue [7]. Agent-based models have also been applied at the level of individual stores. Personalized discounts, as opposed to offering identical discounts to all loyalty members, can positively impact customer satisfaction [8]. Pricing strategies have been compared [9], and queuing systems have been simulated to minimize the time customers spend waiting at checkout [10].

Moreover, satisfied customers can contribute to increased customer flow, which in turn can positively affect store sales [11]. Terano et al. [12] point out that customer flow and sales depend on store layout, and that layout is linked to customer density in the store [13]. Naturally, grocery store owners aim to attract as many customers as possible, i.e., to maximize customer flow to increase revenue. However, excessively high customer density can negatively affect the shopping experience [14], which may ultimately deter customers and negatively impact sales.

There is thus support in the literature for the usefulness of agent-based models in the grocery trade. In some cases, it may also be time-consuming, costly, dangerous, or unethical to intervene in a real system to study the effects of such interventions on system behavior or performance [15]. In such cases, an agent-based approach may be preferable, especially given the ever-increasing

computational power of modern computers and the availability of "Big Data," which can help calibrate and validate newly developed models [16]. To the best of our knowledge, no studies have used agent-based modeling to investigate how customer satisfaction and revenue are affected by the layout of a grocery store. Furthermore, previous studies highlight a lack of understanding of the impact of block layout on store performance [17] or how it can be improved [18]. We therefore propose a model that, given a specific store layout, estimates customer satisfaction and store revenue, with the goal of optimizing both sales and customer satisfaction.

1.2 Purpose and Objectives

The purpose of this study is to implement an agent-based model that can be used to analyze how different block layouts affect customer satisfaction and store revenue. The model is developed according to the design science research framework defined by Peffers et al. [19]. The ODD protocol [20] is used to conceptually formulate the model, which is then implemented in the NetLogo framework [21]. The proposed model is intended to serve as a decision-support tool for planning in the grocery trade, where different section arrangements can be tested on an artificial population to hopefully provide relevant insights and guide decision-making before changes are implemented in the real system.

1.2.1 Research Gap

The lack of knowledge and understanding of how layout affects store performance has previously been noted as limited [22, 18]. Previous studies have examined the layout's impact on impulse purchases [23, 24] or perceived crowding [25, 26], but according to the authors' research, there is a lack of knowledge about how layout affects customer satisfaction with the shopping experience. Given the positive correlation between satisfied customers and profitability [27], and the fact that store atmosphere can influence the shopping experience [28], there is reason to explore how customer satisfaction is affected by store layout.

1.2.2 Research Question

The research question this study aims to answer is: How can an agent-based model be used to investigate the impact of grocery store block layouts on store revenue and customer satisfaction?

1.3 Use of Generative AI

During the development of the model created for this study, Microsoft Copilot was used to generate code snippets [29]. Additionally, Copilot was used as a tool to structure the text, rephrase for improved readability, and translate parts of the referenced texts. All generated code was reviewed and edited, and the developers, who are also the authors, take full responsibility for the code published in connection with this text.

1.4 Structure of the Text

The remainder of the text is structured as follows: Section 2 presents the literature review. Section 3 describes the Design Science methodology applied to develop the model. Section 4 presents the model design, the simulation experiments conducted, and the results. Finally, Sections 5 and 6 are dedicated to discussing the significance of the results, drawing conclusions, and reflecting on areas for improvement and future research.

Kapitel 2

Literature Review

This section presents the conducted literature review. We begin by outlining Wilensky's [30] rule of thumb for determining the usefulness of an agent-based approach. This is followed by a summary of previous agent-based studies on grocery stores. Subsequently, studies addressing the floor plan problem are summarized, followed by studies investigating the concept of satisfaction. Finally, reflections are made, and the identified research gap is pointed out.

2.1 Agent-Based Modeling

As mentioned in the introduction to this text, one of the key advantages of agent-based modeling is that it allows us to study complex systems with a focus on individuals and the choices these individuals make within the system. Wilensky [30] provides a rule of thumb consisting of two criteria that can be used to determine whether a research question may be worth addressing using agent-based modeling. First, there should be a reasonable number (ranging from tens to millions) of individuals or agents interacting with each other or with the environment in which they are situated. In the context of grocery retail, the shopping customers and staff in a store can be viewed as agents, and the store premises as the environment in which they act. Second, agent-based models are useful in situations where the population is heterogeneous, especially when this heterogeneity may influence the behavior or state of the system.

In the context of grocery and retail, several studies have employed agent-based models and simulations. Terano et al. [12] present a simulation study that investigates customer movement and behavioral patterns in a grocery store. The study examines how the store's layout can affect customer flow and how the placement of advertisements impacts sales. It was found that customers' movement patterns depend on the store layout and the placement of advertisements and promotions, suggesting that a store's revenue is influenced by its layout since increased customer flow is linked to increased sales [11]. Furthermore, Baydar [8] demonstrates through simulation experiments that personalized discounts can lead to more satisfied customers and increased sales volumes, which in turn can lead to higher profits. A similar study [9] presents an agent-based model that analyzes pricing and discount strategies. Another issue discussed in this context is queue formation and queuing systems. Zhang [10] develops and implements an agent-based model to simulate and analyze queuing systems in a grocery store and devises an algorithm that is used to reduce aver-

age waiting time and thus increase customer satisfaction. Considering the results of these studies, combined with Wilensky's rule of thumb from the previous paragraph, there is reason to consider agent-based models useful in this context as well.

2.2 The Layout Problem

Studies addressing the floor plan issue approach the problem with varying methods and objectives. In a study from 2000 by Turley and Milliman [22], it is pointed out that this dimension is underrepresented in the literature and further research is suggested (for example, the effect of the layout on perceived crowding or time spent in the store). Since then, some studies have been conducted in this area, but the knowledge is still considered limited [18]. Among previous research, there are studies aimed at maximizing time spent in the store to increase the likelihood of impulse purchases [23, 24], minimizing logistical tasks such as transporting goods from the warehouse to store shelves [31], grouping similar product types together to make the shopping experience as smooth as possible for the customer [18], reducing perceived crowding [25, 26], or distributing customers more evenly throughout the store [13]. The results of these studies show that informed changes to the store layout can have positive effects in several areas. The optimized layout presented by Liu et al. [31] not only reduces logistical intensity but also improves the placement of departments relative to each other and results in less intense customer flow. Although these studies do not focus on the concept of satisfaction, they provide relevant insights that can be connected to it. Thus, they are also helpful for designing and implementing models used to evaluate how different layouts affect customer satisfaction and are therefore taken into account in the development of the model in this study.

2.3 Customer Satisfaction

Customer satisfaction has been widely researched in the past. Consumers' assessment of their own satisfaction can vary depending on, among other things, socio-demographic and cultural affiliation. However, [32] identifies several factors that are highly valued regardless of the country the consumer resides in (the countries compared were the USA and Spain). One of these factors is part of the atmospheric dimension, which supports the relevance of further understanding this factor's significance. This also aligns with results from other studies [33, 34]. The store atmosphere greatly affects how customers perceive the shopping experience [28, 35], which in turn influences customer behavior in the store [11].

In this context, atmosphere is defined as music, color, and store layout, and when these elements influence the customer and put them in a positive affective state, they tend to spend more time in the store, spend more money, and indicate an intention to return to that specific store. A well-planned and logical layout can also make it easier for the customer to navigate the space and find what they are looking for, thereby facilitating the completion of their shopping task. This can contribute to the customer attributing positive values to the shopping occasion [35].

To specify what influences customer satisfaction in grocery stores, several studies have been conducted and have defined a number of critical factors that influence the customer's perception of the shopping experience [36, 37, 32, 38]. Among the factors that affect a customer's satisfaction

are pricing, product assortment, service, quality, and atmosphere [37, 32]. Moreover, a positive correlation has been established between satisfied customers and profitability [27, 39, 40]. [33] also conclude that layout and functionality correlate positively with how the consumer experiences the store environment. Due to the correlation between customer satisfaction and profitability, there are thus strong incentives for actors in the grocery retail market to prioritize customer satisfaction. Furthermore, the significance of atmosphere for the shopping experience (and, consequently, customer behavior in the store) reinforces the relevance of gaining deeper insights into how store layout can make customers more or less satisfied with their shopping experience. As this study aims to investigate customer satisfaction as a factor in simulations, this research is highly relevant.

2.4 Summary and Research Gap

Agent-based models have been found useful for analyzing various aspects of the grocery retail sector, both at the level of individual stores [8, 12, 9, 10, 13] and on a larger scale [6, 7, 5]. Additionally, the placement of departments, shelves, or advertisements has been noted to affect various system behaviors (i.e., store performance), such as increased revenue from a higher number of impulse purchases [23, 24] or reduced congestion [25, 26]. Combined with the finding that atmosphere is significant for how the shopping experience is perceived [28, 33], and the positive relationship between satisfied customers and profitability, it is of interest to be able to analyze a store layout with customer satisfaction in mind. Despite this, there is currently a lack of studies utilizing agent-based models to examine the concept of satisfaction, which could also be a way to increase revenue and strengthen competitiveness. Furthermore, an agent-based model can provide a tool for analyzing and comparing different design decisions before they are implemented.

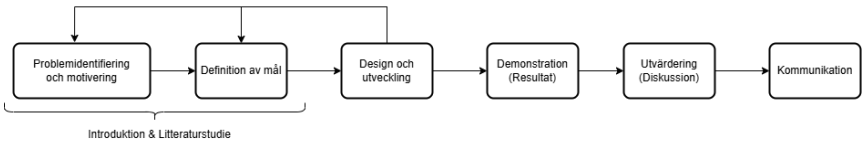
Kapitel 3

Methodology

This study describes and evaluates an agent-based simulation of how a grocery store's layout affects its customers and revenue. Within Design Science, new knowledge is generated by designing and evaluating artifacts, represented by constructs, models, methods, and instantiations [41, 42]. This means that research generates material or theoretical results in one of these four forms. The artifacts aim to achieve a goal or solve a problem.

3.1 Design Science Research Methodology

The study is conducted in accordance with the Design Science Research Methodology (DSRM) [19]. DSRM is divided into six activities, which in this study have been carried out iteratively, inspired by the cycles defined by Hevner [43]. Figure 3.1 illustrates the activities of the process and their order. This section describes DSRM and its six activities, as well as where the results of each activity can be found in this text.



Figur 3.1: Overview diagram of the process

1. **Problem identification and motivation**
   Identify and define the problem the research aims to solve and justify why a solution to this problem would be valuable [19]. This requires knowledge about the problem and the importance of a solution. This activity is described under heading 3.2.

2. **Definition of the objectives for the solution**
   Define objectives for a solution by referring to the problem description and knowledge of what is possible within the domain [19]. The objectives can be qualitative or quantitative and should be derived from the problem. This activity may result in explicit system requirements for the solution. This activity is described under heading 3.3.

3. **Design and development**
   Artifacts are created, which can be categorized into the following: constructs, models, methods, or instantiations [19]. Generally, an artifact is a designed object whose design includes contributions to research. In this activity, the desired functionality and architecture of the artifact are defined, and the artifact is then created. This activity and its results are presented under heading 3.4.

4. **Demonstration**
   The usefulness of the artifact is demonstrated by solving one or more instances of the defined problem [19]. The demonstration is presented under heading 4.

5. **Evaluation**
   The artifact is evaluated based on how well it solves the problem identified in activity 1 [19]. The objectives from activity 2 are compared with the results achieved during the demonstration in activity 5. This requires relevant data and analysis techniques. The method for the evaluation is described in 3.5, and the result of the evaluation is presented under heading 4.

6. **Communication**
   The research is communicated through a publication [19]. The problem and its relevance, as well as the following aspects of the artifact, should be discussed and communicated: its usefulness and relevance, its design and effectiveness, and its potential for future research and other relevant audiences. The design of the artifact is communicated using the ODD description in Appendix A and discussed in the following sections 5 and 6.

3.2 Problem Identification & Motivation

The problem definition is based on previous research conducted in the same field. In order to define the problem and formulate a relevant research question, the existing literature in the area was reviewed. The purpose of this is threefold. The first purpose is to identify the problem and formulate the research question. In addition, the review is intended to support the implementation of the actual model by contributing to an understanding of which factors may be relevant to investigate in relation to the stated question. Finally, the literature review results in chapter 2, where the current research landscape and research gap are described, and the stated research question is justified and placed in context. The literature is reviewed following a simplified version of the process described by Oates [44], where the review of existing literature was conducted in four phases. These are: searching for sources, evaluating the relevance of the sources, reading the identified sources that may be relevant, and finally compiling the conclusions of the articles.

3.2.1 Identifying sources

The goal of this phase is to identify as many texts as possible that relate to the research question forming the basis of this study. To make the search for relevant literature more efficient and to minimize the time required, only Google Scholar was used. In a comparison between six databases [45] (the compared databases were Google Scholar, Microsoft Academic, Scopus, Dimensions, Web of Science, and OpenCitations' COCI), Google Scholar was found to be the most comprehensive search engine among those compared. As mentioned above, the purpose of the literature review

is to obtain an overview of the current research landscape, identify research gaps, and formulate a relevant research question.

3.2.2 Evaluation of the Identified Sources

Once sources have been identified, an initial filtering is done based on each article's abstract. If any of the inclusion criteria are met, the text proceeds to the second selection phase, where it is read in full. The criteria are:

- the text concerns agent-based modeling applied to the grocery retail sector,
- the text concerns agent-based modeling and/or customer satisfaction,
- the text investigates how customer satisfaction is affected by one or more factors,
- the text specifically concerns the relationship between customer satisfaction and atmosphere,
- the text concerns agent-based modeling and layout,
- the text concerns customer satisfaction and layout,
- the text specifically concerns layout.

3.2.3 Review and Compilation of Sources

Once the initial filtering is complete, the relevant texts are read in full and then summarized and compiled. The texts are analyzed based on seven components, which are: problem identification, objectives, research method, weaknesses or limitations, results, conclusions, and suggestions for future research or improvements.

3.3 Definition of Objectives

The goal of this research is to create a simulated grocery store where different layouts can be tested based on how they affect the store's revenue and the simulated customers' satisfaction with their experience. This means that the model should provide a way to simulate different layouts of a grocery store and that data on revenue and customer satisfaction should be presented once the simulation is completed. This is to allow for an evaluation of the layout based on these two factors.

In order for the model to fulfill its purpose, several criteria must be met: there must be different layouts available for simulation, the user must have the option to design their own layout, there must be other necessary infrastructure in the store (e.g., checkouts, entrance, exit) to enable a simulation, and there must be customers who interact with the store layout. Under heading A.1 Purpose and patterns, the model's purpose is specified in more detail, where a collection of patterns—observed in previous research and in real life—that the model is based on can also be found.

Previous models of stores have been developed [46, 47, 12, 13, 23]. These models represent several aspects of grocery stores, including queuing systems, customer behaviors, and even layout optimization, which this model also aims to address. However, this model fills a gap by describing stores

based on their department layout—something that has not been done previously. Furthermore, the simulation presents data on customer satisfaction and revenue, which can be used for analyzing different layouts.

3.4 Design and development

3.4.1 The ODD protocol

The conceptual model is formulated and described according to the ODD protocol [4]. ODD (overview, design concepts, details) aims to provide a clear and comprehensible overview of agent-based models in a standardized way. In addition, the ODD protocol intends to facilitate the implementation of the conceptual model in any programming language as well as to replicate it, in order to evaluate its correctness or further develop it [4, 48]. Several of the elements in ODD correlate with activities in DSRM and are briefly described under each respective subsection; see appendix section A for the full ODD description of the model.

3.4.2 NetLogo

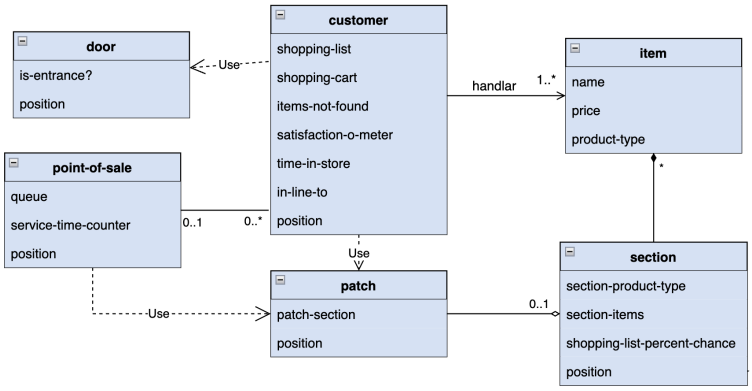
The model and the simulations that are the subject of this study are implemented using the NetLogo framework [21]. NetLogo is a widely used programming language built on Java that is used for simulations. NetLogo provides a graphical interface to visually present simulations and is designed specifically for simulations, offering semantics that simplify, for example, the handling of agents. It would not have been impossible to implement the model in another programming language with different features, but this would have required developing the graphical elements from scratch, which would have taken time away from the work on developing the model. Besides the fact that NetLogo abstracts away these visual details (graphical interface, buttons, sliders, graphs, etc.), there is also an established practice of conducting simulations in NetLogo, which has resulted in a large user community and extensive documentation.

3.4.3 Previous research as a framework

The development of the model has been based on the previous research presented in section 2 Previous research. This concerns attributes and behaviors of agents as well as how different procedures have been designed. During development, requirements were collected from previous research and ranked according to relevance and how well the requirements contribute to achieving the goal of the model, and then a selection was made also based on estimated implementation time. Justifications for the choices made during development can be found in the ODD protocol under the relevant section where the decision was made.

3.4.4 The model

The model is described in its entirety according to the ODD (overview, design concepts, details) protocol in the appendix under heading A ODD description of the model [4].

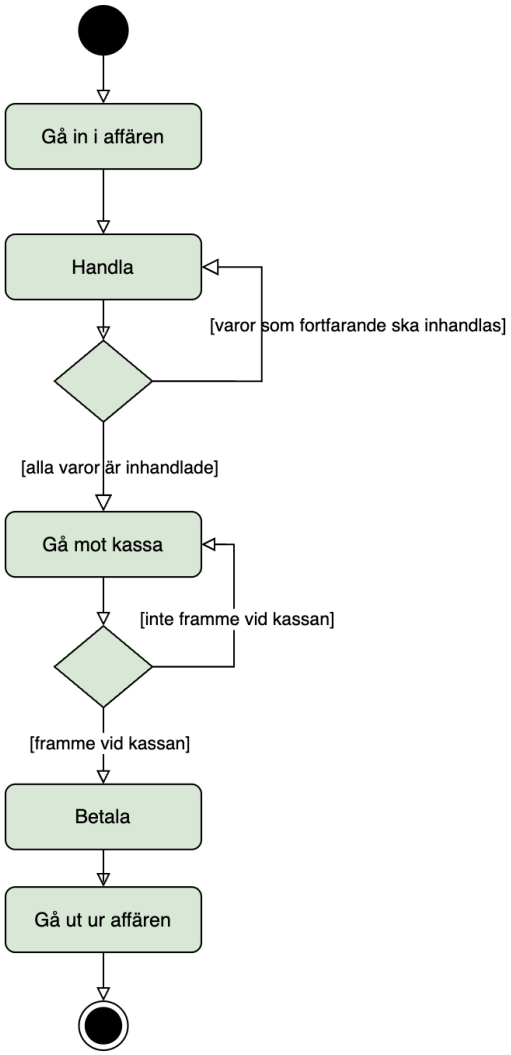


Figur 3.2: Entity relationship diagram of agents in the model

Structure

The model consists of a NetLogo file with code and comments. The file includes 4 predefined layouts that the user can choose from, as well as the option to create their own layout in the code [21]. The model consists of 6 entities described in Figure 3.2. The diagram depicts the relationships between the simulation's different entities according to the UML standard, where the instance variables in the diagram correspond to the entities' state variables [49]. A customer has several items on their shopping list and in their cart, and they have a reference to a checkout when they have queued for it. Customers use a door entity to position themselves at the entrance when created and to move toward the exit at the end of their lifecycle, and they use patches to check whether they are standing in a department and if so, which one. A patch represents zero or one department, and patches are also used by checkouts to position themselves. Finally, each department has a list of items available in that department.

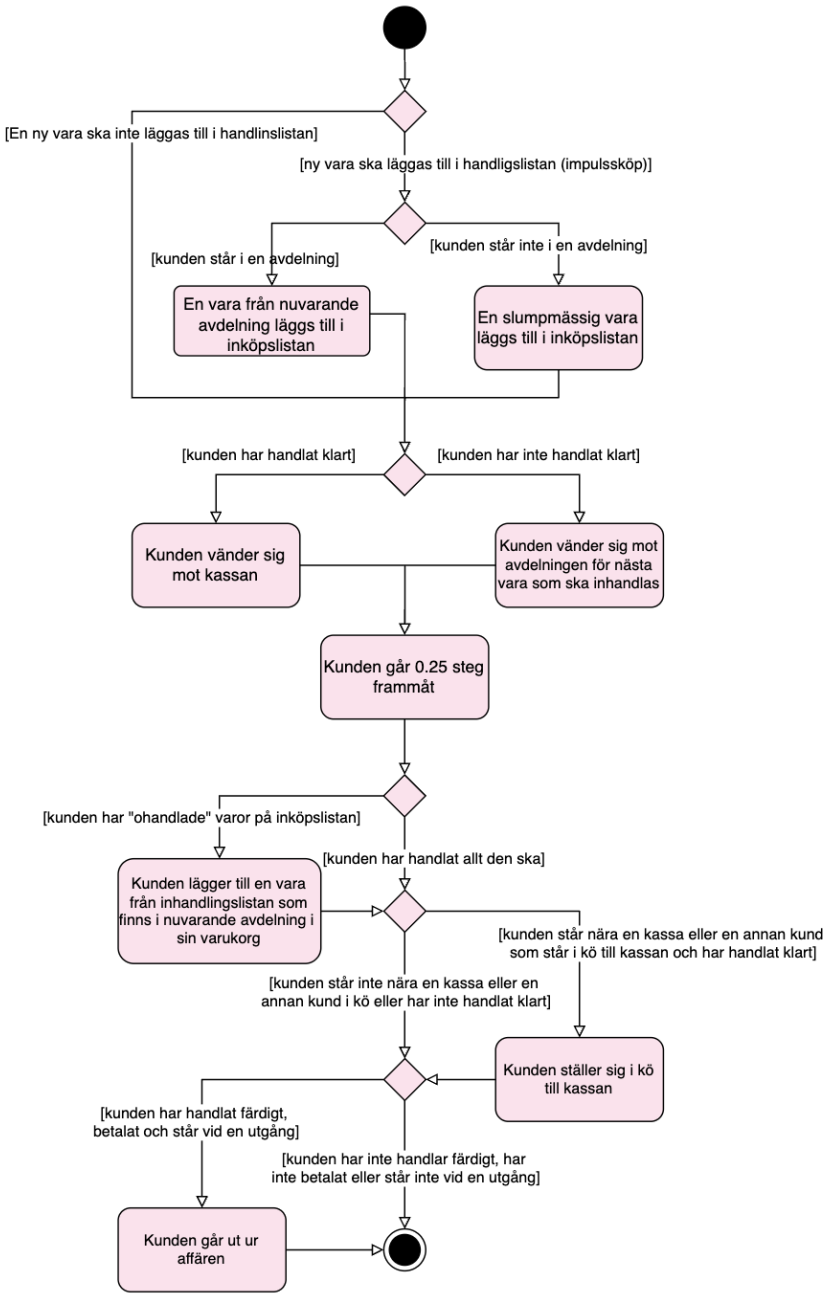
Customers shop for items in the store. These items are located in departments which are placed on different patches, i.e., "cells," in the store. When a customer has picked all the items they need, they pay at a checkout and leave the store.



Figur 3.3: Activity diagram: The customer's life cycle

Functionality

The model enables simulations of customer behavior in a grocery store with the chosen layout. When the user starts a simulation, at each "tick," a "go" procedure is executed which is responsible for the entities performing their desired behaviors. The simulation continuously presents the store's total revenue since the start and the total satisfaction of customers who have left the store. A customer's lifecycle in the simulation from the moment they enter the store until they leave is described in Figure 3.3. The simulation proceeds in increments of the time unit "tick," and at each tick, several procedures are executed. This includes all customers running a procedure described in Figure 3.4, all checkouts managing their queues, and the possibility of a new customer being



Figur 3.4: Activity diagram: One time unit for a customer

created and entering the store.

In Figure 3.3, a customer agent's lifecycle in the simulation is described. The customer is created at the store entrance with an empty cart and a shopping list; then the customer shops, meaning walking around between different departments in the store until all items on the shopping list are found. Then the customer goes to a checkout and queues if there is any customer ahead, pays, and moves toward the exit. When finally reaching the exit, the customer is removed from the simulation.

In Figure 3.4, a step in the simulation for a customer is illustrated. The process begins by evaluating whether an impulse purchase should occur, and if so, a new item is added to the customer's shopping list. Then, the customer's direction is adjusted depending on whether they are to continue shopping—they turn toward the department they want to buy an item from—or if they have finished shopping, they turn toward a checkout or the exit. Then, the customer searches for items to buy at their current location if they still have items to purchase. If the customer has completed their shopping, it is checked whether the customer is close enough to a checkout or another customer in the queue to join the checkout line. Finally, it is checked whether the customer has finished shopping and is at the exit. If so, the customer is removed from the simulation and the procedure ends.

Use plan

The model can be used to investigate how a potential layout of a grocery store affects its revenue and customer satisfaction. The data output by the model should not be seen as actual measures of revenue or customer satisfaction but rather be used for comparative and exploratory purposes. This means the model is best suited when two or more potential layouts are to be compared. Users can run simulations of the existing layouts (examined in this study) or build their own layout in the procedure *setup-custom-layout* using *setup-one-section-patch*. Then, a suitable number of different simulations should be run to counteract outlying or deviant results caused by the program's random elements.

3.5 Evaluation

To validate the model, results from different layouts are compared with each other based on the perspective and assumption that layouts designed according to patterns observed in reality should yield good results. This is based on the assumption that real grocery stores have a profit motive and thus should be designed with the aim of generating revenue, so the model should show high revenue values for layouts that reflect real stores. Furthermore, satisfied customers benefit stores both in individual purchase occasions and over longer periods, as customers tend to return to stores where they have had good experiences. This indicates that these types of layouts should also produce good results for overall satisfaction [27, 39, 40]. The evaluation is therefore conducted by examining the results from layouts designed to resemble reality compared to layouts that contradict patterns observed in real stores. If the first type shows better results than the latter, this suggests that the model fulfills its purpose and works to answer the research question investigated in the study.

Kapitel 4

Results

The results of the research are twofold. First, the artifact created during the research is presented in accordance with the structure presented by Johannesson and Perjons in section 1.10 [41]. In addition, the results of a number of simulations are presented and compared.

4.1 Source Code

The research resulted in the model described in section 3.4.4 Model and in A ODD-description of the model. The model's source code is published in NetLogo's community library and is available at the following URL: Note: We are waiting for the code to be uploaded to the NetLogo website. An URL will be given in the final report

4.2 Layouts

The NetLogo program has four built-in layouts and the user has the option to create their own layouts to explore these. Each layout consists of 12 departments represented by differently colored blocks; all included layouts follow the color scheme described in the table below. The table represents layout 1 in figure 4.1, position and color reflect the department located at the corresponding position and color in the layout. The table does not consider the other components found in the layout. Besides the departments, at simulation start, there are also two checkouts, represented by a red square, one entrance represented by a smaller green square, and one exit represented by a smaller red square. In all four layouts, these are located along the model's southern edge.

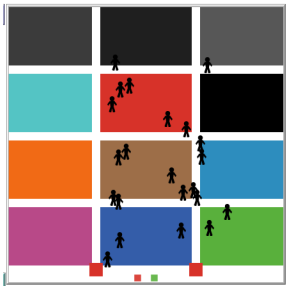
4.2.1 Layout 1

Description

The layout is depicted in figure 4.1. Its departments are placed in a grid with 3 columns and 4 rows. Row 1 consists of departments, from right to left; Children, Hygiene, and Pets, row 2; Drinks, Meat, and Household, row 3; Pantry, Bread & Bakery, and Freezer, and finally, row 4; Snacks, Dairy, and Fruit & Vegetables. The checkouts are placed between the departments in row 4 at y-coordinate

Tabell 4.1: Color and department scheme for layout 1

Row	Column:	A	B	C
1	**Color:**	Medium gray	Dark gray	Light gray
	Department:	Children	Hygiene	Pets
2	**Color:**	Turquoise	Red	Black
	Department:	Drinks	Meat	Household
3	**Color:**	Orange	Brown	Light blue
	Department:	Pantry	Bread & Bakery	Freezer
4	**Color:**	Pink	Dark blue	Green
	Department:	Snacks	Dairy	Fruit & Vegetables



Figur 4.1: Layout 1

-15 and the entrance and exit are located south of the dairy department (blue, middle of row 4), the exit (red) to the right and the entrance (green) to the left.

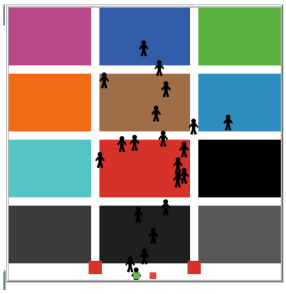
Rationale

Layout 1 is built on the principle that all 12 sections take up the same space in the store; this was done to enable testing where customer behavior is not influenced by the size of the department. The departments are placed according to the probability that a customer wants to buy something from the department, with the most popular departments closest to the checkouts. The customer's shopping list is randomized based on data about which departments customers most frequently shop from (see A.7.10 *randomize-shopping-list* for more information about the process); the same data has been used in this simulation to place the departments [50]. The most "popular" department is the dairy section (blue) and is placed in the middle closest to the entrance (position B4) and the least popular department, Children, is placed farthest from the entrance at position A1.

4.2.2 Layout 2

Description

Layout 2 is shown in figure 4.2. Its departments are placed in a grid with 3 columns and 4 rows. Row 1 consists of departments, from right to left; Snacks, Dairy, and Fruit & Vegetables, row 2; Pantry, Bread & Bakery, and Freezer, row 3; Drinks, Meat, and Household, and finally, row 4;



Figur 4.2: Layout 2

Children, Hygiene, and Pets. The checkouts are placed between the departments in row 4 at y-coordinate -15 and the entrance and exit are located south of the hygiene department (dark gray, middle of row 4), entrance (green) to the right and exit (red) to the left.

Rationale

Layout 2 is built according to the same pattern as layout 1, and all 12 departments take up equal space here as well; the difference between the two is the placement of the departments and the entrance and exit. The departments follow the opposite pattern from layout 1 and instead are placed with the least popular department, the hygiene section (dark gray), closest to the entrance and the most popular dairy department (blue) farthest from the entrance, with the remaining departments arranged according to the principle that the more popular, the farther from the entrance. This is based on the same study motivating layout 1 [50].



Figur 4.3: Layout 3

4.2.3 Layout 3

Description

Layout 3 is shown in figure 4.3. The departments are placed in approximately two columns with some exceptions for smaller departments. The layout's flow follows an upside-down horseshoe with the entrance at the right end and the exit at the left. The entrance and exit are located along the model's southern edge, with the entrance (green) to the right and the exit to the left; the checkouts

are placed just northwest of the exit. The departments are, from the entrance's perspective (right bottom edge) and along the model's right side: Fruit & Vegetables, Meat, Bread & Bakery, and Pantry at the top right. At the top left is the dairy department, and south of it on the same latitude are the children's department to the left and household to the right. Below them are the drinks department, followed by pets and hygiene departments again on the same latitude as each other. Finally, near the bottom left is the freezer department and south of it the snacks department, which is the department closest to the two checkouts and the exit.

Rationale

The layout is built to resemble a grocery store in central Malmö as it looked in April 2025. Some simplifications have been made in the model of the store. The purpose of the layout is to test the model on how well it reflects reality in terms of how customers behave in a store taken from reality.



Figur 4.4: Layout 4

4.2.4 Layout 4

Description

The layout is shown in figure 4.4. Its entrances and checkouts are placed in the same way as in layout 3. The departments follow no pattern for either size or placement. At the top of the layout are the pantry and dairy departments, below the orange pantry department are the drinks (turquoise) and snacks (pink) departments next to each other, and south of these is the freezer department (light blue). In the middle of the store is the fruit & vegetable department (green) and south of it the bread & bakery (brown). Along the layout's right edge, south of the dairy department, are, from top to bottom, household, hygiene, children, and pets. In the bottom right corner is the meat department (red).

Rationale

This layout was used during the construction of the program and has served its purpose during the development of the model. Compared to the other layouts, there is no clear structure or reasoning behind this layout, which is used during the model validation to have a random and unstructured layout to compare the other more thought-out layouts against.

4.3 Simulations

4.3.1 Structure

All built-in layouts were simulated in this experiment. A simulation means that the *setup* procedure is run once and the *go* procedure is run 5000 times consecutively. This is to simulate 21 hours in the grocery store since each *tick* represents 15 seconds in the store and $15 \cdot 5000 = 75\,000$ and $\frac{75\,000}{60^2} \approx 21$, which approximately corresponds to 2 days in a grocery store. Each layout was simulated 50 times with the possibility to run 6 simulations in parallel to obtain a general result relatively quickly. The results of the runs are found under heading 4.3.2 Results.

4.3.2 Results

Tabell 4.2: Average simulation results of 50 runs of 5000 *ticks* for the 4 layouts

Layout	**Average revenue**	**Standard deviation revenue**	**Average customer satisfaction**	**Standard deviation customer satisfaction**
1	1 788 538	284 542	-47 340	10 578
2	1 689 135	263 860	-37 287	9 980
3	2 102 515	338 053	-80 463	31 908
4	1 343 505	206 591	-110 295	44 421

The results in table 4.2 are rounded to the nearest integer; customer satisfaction resulted in negative values for all simulations. The values produced by the simulations represent, for revenue, the total cost of the goods that customers successfully purchased, and satisfaction is the sum of the total satisfaction of all customers who left the store.

The results show that layout 1 has the second highest value for both revenue and customer satisfaction, layout 2 has the second lowest value for revenue but the highest value for customer satisfaction, layout 3 has the highest value for revenue and the second lowest value for customer satisfaction, and finally layout 4 has the lowest value for both revenue and customer satisfaction. The averages for revenue lie between 1,343,505 and 2,102,515, which gives a difference of 759,010. The standard deviation for revenue is fairly uniform and corresponds to the revenue values. The relative standard deviation, calculated from the values presented in table 4.2, for all layouts lies between 15.4% and 16.1%. The values for average customer satisfaction lie between -110,295 and -37,287, which gives a difference of 73,008. The standard deviation for customer satisfaction varies considerably more than for revenue and lies between 9,980 and 44,421 with a relative standard deviation between 22.3% and 40.3%.

4.4 Model Validation

To answer the research question from 1.2.2 Research Question, an evaluation of the model's validity is required. The model aims to reflect reality and can be considered to fulfill its purpose, and thereby answer the question of *how* an agent-based model can be used to investigate grocery store

layouts, if it reflects reality.

The validation is based on the assumption that grocery store chains design their stores with the intention to make a profit, which would imply that a simulation with a layout that depicts a real store should lead to a high value of total revenue. In addition, satisfied customers lead to positive consequences for the grocery store such as repeat visits and that the customer spends more time and money during a visit [27, 40, 11, 39]. This indicates that a well-planned store layout should also result in high values for customer satisfaction.

To determine whether the model fulfills its purpose and to examine its connection to reality, layouts 3 and 4 are used. Layout 3 is created to resemble a real grocery store and layout 4 is created not to follow such patterns. The comparison between the two layouts shows that layout 3, i.e., the real grocery store, performs better than layout 4 in terms of both revenue and customer satisfaction. Layout 4 shows the worst results of all layouts in the model both in terms of revenue, average 1,343,505, and customer satisfaction, average -110,295. Layout 3 ranks best regarding revenue with 2,102,515 on average and second worst in terms of customer satisfaction with -80,463. Regarding revenue, the difference between the layouts' averages is 759,010, which is a marked difference and indicates that layout 3 is better at generating revenue than layout 4. The same applies to customer satisfaction where the difference is 29,832, which is not as large as for revenue but is still relevant for the comparison.

Kapitel 5

Discussion

This chapter discusses the results obtained from the simulation experiments, followed by reflections on the weaknesses and limitations of this study as well as of the implemented model.

5.1 Simulation Results

The results from the simulations show that layout 4, which was designed without any particular strategy (see 4.2.4), achieves the lowest results both in terms of revenues and customer satisfaction. Layout 3 (which imitates a real grocery store, see 4.2.3) achieves the highest average revenue, and layout 2 (see 4.2.2), which places the most visited departments furthest from the entrance, has the highest average customer satisfaction. Layout 1, where the most visited departments are placed closest to the entrance (see 4.2.1), achieved the second highest results in both customer satisfaction and revenue.

That layout 4 produced the worst results was expected since it is neither based on previous research nor patterns observable from reality; thus, there is no evidence supporting that it would be a successful grocery store design. The highest revenues were observed in the simulations conducted on layout 3, which aims to replicate a real grocery store. This result is also as expected based on the logic that a grocery store intending to be profitable should have a well-planned layout to fulfill this purpose. What, however, is surprising is the discrepancy between customer satisfaction and revenues in layouts 1–3. This is because there is a known relationship in the literature between satisfied customers and high revenues and because the likelihood of impulse purchases increases the more satisfied a customer is in the model [27, 39, 40]. The results show that layout 3 with the highest revenues also had the second lowest customer satisfaction, and that layout 2 with the highest customer satisfaction has the second lowest revenues. One factor that could explain this is that all customer satisfaction results are negative. From this, the conclusion can be drawn that many customers also provide many negative customer satisfaction values which are added together and produce a large number for total customer satisfaction. Thus, one reason for this discrepancy may be that the more customers flow through the store, the higher the revenues but also the lower the total customer satisfaction. However, the results from layout 4, which had both the worst revenue and customer satisfaction values, contradict this theory.

Layout 1 and 2 have the same design except that the departments are placed in reverse order, so in layout 1 the most visited departments (based on how the customer agents' shopping lists are generated) are placed closest to the entrance, exit, and checkouts, while in layout 2 they are placed furthest away. The difference in results between the two can thus be an interesting report on the model's functionality. That layout 1 had higher revenues but lower customer satisfaction than layout 2 is surprising since the model is designed so that customers should be satisfied spending relatively short time in the store, and this should occur with a layout where popular departments are close to the entrance and exit. Moreover, a customer spending a long time in the store should result in more impulse purchases, which would lead to higher revenues and happen as a consequence of a customer having to move far into the store as in layout 2. On the other hand, this could also be a result of the effect described above, that high revenues indicate a layout with a high customer flow and that many customers in turn lead to lower customer satisfaction scores because the satisfaction values are generally negative.

5.2 The Model as a Result

The research question posed in the introduction reads: *How can an agent-based model be used to investigate grocery stores' layouts impact on the store's revenues and the shopping customers' satisfaction?* To answer this, the model in this study was created. To determine whether the model is a sufficient answer to the question, layouts 3 and 4 were utilized and simulated with results shown in 4.4 Validation of the Model. The results show a large difference in values produced by the different layouts; layout 4, with departments placed without strategy, shows the lowest values among all layouts in both categories, and layout 3, which is intended to imitate a real grocery store, achieved the highest revenues and better customer satisfaction than layout 4. From this, the conclusion can be drawn that the model is a possible solution to investigate grocery store layouts' impact on its revenues and customer satisfaction and thus an answer to the research question.

The model has weaknesses that prevent it from fully fulfilling its purpose, which are discussed below in 4.4. The model validation is based only on two layouts compared, and ideally should be explored further on several different types of layouts. The conclusion that can be drawn is that these results indicate that the model is a tool to investigate and compare different layouts producing values in line with reality, but from these results, it cannot be said how accurate the model is.

5.2.1 Use of the Model

The model only fulfills its purpose and can be considered an answer to the research question if it is used correctly. The user must keep in mind that the model is not a direct representation of reality and use it in accordance with the instructions described in section 3.4.4 Usage Plan. If these are not followed, there is a risk that incorrect conclusions will be drawn from the model's results. Since the values for both revenue and usability are arbitrary and only intended to generate a value for comparison.

5.3 Limitations and Weaknesses

This section is dedicated to reflections on the weaknesses and limitations of this study. The reflections concern the lack of Big Data collections, the complexity surrounding the concept of satisfaction, the ways in which the model can be considered incomplete, as well as weaknesses related to the conducted literature review.

5.3.1 Big Data

Regarding this study's limitations and weaknesses, several areas for improvement exist. The absence of "big data" can be seen as one. The studies conducted by Sturley et al. [6], Schenk et al. [7] and Pantano et al. [13] were done in cooperation with either individual stores or larger actors on the grocery market. With these collaborations, the authors had access to various data collections, e.g., from membership data, sales statistics, or receipts. Such data collections can be useful during the design and implementation of the model. Through analysis of such datasets, there may be opportunities to gain a better understanding of purchase behaviors or patterns [16], and thereby make assumptions that better correspond to reality. Furthermore, there may be reasons to use some data collection (e.g., receipts) as input to simulations run using the model.

5.3.2 The Complexity of Customer Satisfaction

Furthermore, a natural weakness is the fact that many factors influence satisfaction, and these factors have different weights for different individuals. Several studies point out the complexity of analyzing satisfaction due to the number of factors that can affect how satisfied a customer perceives themselves to be with the service used. Among these factors are, for example, product assortment, product quality [32, 27], opening hours, proximity to home, and service [32]. In addition, it is noted that the concept of satisfaction depends on, among other things, cultural and socio-demographic affiliation, which results in the same factor having different impacts on perceived satisfaction depending on, for example, which cultural group the customer belongs to [32, 37] or which type of store they usually visit [37]. Moreover, modern consumers' behaviors and shopping patterns have been shown to have become more complex, which means this may vary depending on socio-demographic affiliation [32] or shopping goals [6, 36]. Furthermore, a customer's evaluation of their satisfaction with the experience can be divided into three phases (before, during, and after) [27], where the customer first forms their expectations, then interacts with the service, and finally weighs the initial expectations against the final result of their interaction with the service used. Reasonably, things can happen even after the shopping occasion that negatively affect the customer's satisfaction (e.g., discovering poor quality or condition of a product after the purchase), but the model does not take this into account and only considers the during phase.

Additionally, a weakness relates to the consumer's purpose, i.e., the reason the consumer uses the service in question. In this regard, consumption can be divided into two camps depending on whether the consumption provides utilitarian or hedonic value [51, 36]. Both perspectives can reasonably play a role for grocery consumers, as, for example, cooking enthusiasts may perceive it as pleasurable to explore the product assortments from different suppliers. Moreover, Holmgren [24] highlights that a significant portion of US grocery retail revenues come from impulse purchases, and that 75% of customers report that some purchase decisions are made during the shopping occasion.

Due to the scope and timeframe of this work, this model only takes into account the utilitarian perspective (i.e., the customer is satisfied if they manage to purchase all the items on their list and less satisfied if not).

5.3.3 Model Incompleteness

Additional weaknesses of the model include the absence of mechanisms and processes on the supply side of the grocery retail chain. Considering the timeframe of this work, only the consumer's perspective and experience of the shopping occasion were included, but it may still be relevant to incorporate processes related to the store's staff. One example of this could be the transportation of goods from the warehouse to the department or store shelf. As pointed out by Liu and Liu [31], the optimized layout resulted in reduced logistical intensity (the logistical work became more efficient). This efficiency may also be linked to customer satisfaction, as it can result in fewer situations where customers have to wait for shelves in the store to be restocked. Goić et al. [27] note that negative experiences often weigh more heavily than positive experiences when customers evaluate their satisfaction, which speaks to the relevance of including both staff and the processes carried out by staff in the model.

5.3.4 Previous Research as a Framework

Given that the constructed model has been implemented with the help of results from previous research, there are also some factors to consider on this level. One of these is the risk that the implemented functions and processes in the model are based on information that itself has certain weaknesses and limitations. This may, for example, be measurements that cannot be generalized, shortcomings in reliability, or a lack of sufficient validation. Evidence-based software development of this kind requires rigorous and carefully conducted evaluation and analysis of the available evidence [52]. That said, there are additional weaknesses in this study relating to the way the literature review was conducted. For example, only Google Scholar was used when searching for sources. This decision was made considering the timeframe and purpose of the work, but expanding the search to include several databases and domain-specific journals might possibly have provided a better foundation on which to base the implementations.

Kapitel 6

Conclusion

6.1 Summary

This study has investigated how agent-based modeling can be used to examine grocery store layouts. The research conducted within the scope of this study shows that the model developed and published in connection with the study is a tool that can be used in situations where grocery store layouts are compared based on how they affect the store's revenues and the shopping customers' satisfaction. The model has development potential and has been tested under few different conditions, but the results from these experiments show a functioning model.

6.2 Implications

The results from this study have several implications, both for future research on the subject and for practical applications in grocery retail.

First, the study shows that agent-based modeling in general, and this model in particular, is a useful tool for conducting simulations and thereby analyzing how different store layouts affect both customer satisfaction and business revenues. This means that the model is suitable to be used as decision support within grocery retail to make informed decisions during the planning of a grocery store layout. Since the model produces multiple results, these can be used as a basis for balanced decision-making that promotes both profitability and customer satisfaction.

From a research perspective, the study and its results imply that agent-based modeling is suitable for several investigations of this kind and

6.3 Future Work

Regarding future work, there are also several directions to pursue. Initially, further development of the model is proposed. By incorporating more relevant components into the model (for example, working staff and other mechanisms from the supply side), the model can represent reality in a more

realistic and complete way, thereby making better estimates of satisfaction and income. Furthermore, a higher degree of interactions between agents can be implemented in the model. Currently, customers only interact with the checkout, but it may also be relevant to implement interactions between customers and between customers and working staff, for example, customers asking staff where a specific product is located.

The customer agents themselves can also be further developed to give more realistic behavior. Examples include that a customer becomes more satisfied by being first in the checkout queue or by spending minimal time waiting in the checkout line. Also, some form of rush hour traffic could reasonably be implemented so that traffic in the store is more intense during certain parts of the day. Considering the weaknesses pointed out in 5.3, customer agents could also be diversified to a greater extent and include agents from different demographic and social groups where these different groups evaluate their satisfaction based on different parameters, have different daily rhythms or shopping preferences. The model can also be extended by implementing the effects of crowding on customer behaviors, for example by increasing the chance of impulse purchases [53].

Developing the model in collaboration with one or more actors in the grocery market could also be a way to improve the model. With the help of a partner, there may be opportunities to access data collections that can be analyzed or provided as input to the simulations. A collaboration can also provide insight into the business and thus contribute perspectives that for various reasons are overlooked in research. Heppenstall et al. [16] highlight that Big Data can be helpful for calibration and validation of agent-based models, but also point out that high-quality and high-resolution data is required to make well-informed assumptions or efficient validation work.

Bibliography

[1] T. C. Schelling. "Models of Segregation". In: *The American Economic Review* 59.2 (May 1969), pp. 488–493. ISSN: 0002-8282. URL: https://www.jstor.org/stable/1823701.

[2] Md Salman Shamil et al. "An agent-based modeling of COVID-19: validation, analysis, and recommendations". In: *Cognitive computation* (2021), pp. 1–12. DOI: 10.1007/s12559-020-09801-w.

[3] T. Marwala E. Teweldemedhin and C. Mueller. "Agent-based modelling: a case study in HIV epidemic". In: *Fourth International Conference on Hybrid Intelligent Systems (HIS'04)*. Dec. 2004, pp. 154–159. DOI: 10.1109/ICHIS.2004.16.

[4] S. F. Railsback and V. Grimm. *Agent-Based and Individual-Based Modeling*. Second edition. Princeton: Princeton University Press, 2019. 340 pp. ISBN: 978-0-691-19082-2.

[5] Jürgen Rauh, Tilman A. Schenk, and Daniel Schrödl. "The Simulated Consumer - an Agent-Based Approach to Shopping Behaviour". In: *Erdkunde* 66.1 (2012). Publisher: Erdkunde, pp. 13–25. ISSN: 0014-0015. URL: https://www.jstor.org/stable/41444860 (visited on 01/22/2025).

[6] Charlotte Sturley, Andy Newing, and Alison Heppenstall. "Evaluating the potential of agent-based modelling to capture consumer grocery retail store choice behaviours". In: *The International Review of Retail, Distribution and Consumer Research* 28.1 (Jan. 2018), pp. 27–46. ISSN: 0959-3969, 1466-4402. DOI: 10.1080/09593969.2017.1397046. URL: www.tandfonline.com/journals/rirr20 (visited on 01/22/2025).

[7] J. T. A. Schenk G. Löffler and Rauh. "Agent-based simulation of consumer behavior in grocery shopping on a regional level". In: *Journal of Business Research* 60.8 (Aug. 2007), pp. 894–903. DOI: 10.1016/j.jbusres.2007.02.005.

[8] C. Baydar. "Agent-based modeling and simulation of store performance for personalized pricing". In: *Proceedings of the 2003 Winter Simulation Conference*. Vol. 2. Dec. 2003, pp. 1759–1764. DOI: 10.1109/WSC.2003.1261630.

[9] N. I. Arvitrida and A. C Kurniawan. "An agent based simulation model for retail price promotion strategies". In: *AIP Conference Proceedings* 2217.1 (Apr. 2020), p. 030154. DOI: 10.1063/5.0000509.

[10] Q. Zhang. "Multi-agent Based Supermarket Queuing Model and Optimization". In: *Advances in Multimedia, Software Engineering and Computing Vol.2*. Ed. by David Jin and Sally Lin. Berlin, Heidelberg: Springer, 2012, pp. 635–640. ISBN: 978-3-642-25986-9. DOI: 10.1007/978-3-642-25986-9_99.

[11] Fon Sim Ong et al. "A comparison between structural equation modelling (SEM) and Bayesian SEM approaches on in-store behaviour". In: *Industrial Management & Data Systems* 118.1 (2018), pp. 41–64. DOI: 10.1108/IMDS-10-2016-0423.

[12] T. Terano et al. "Agent-based In-store Simulator for Analyzing Customer Behaviors in a Super-market". In: *Proceedings of the 13th International Conference on Knowledge-Based and Intelligent Information and Engineering Systems (KES 2009)*. Santiago, Chile: Springer, 2009, pp. 244–251.

[13] E. Pantano et al. "Enhancing Store Layout Decision with Agent-based Simulations of Consumers' Density". In: *Expert Systems with Applications* 182 (2021), p. 115231. DOI: 10.1016/j.eswa.2021.115231.

[14] A. Aydinli. "How Do Customers Alter Their Basket Composition When They Perceive the Retail Store to Be Crowded? An Empirical Study". In: *Journal of Retailing* 97.2 (June 2021). DOI: 10.1016/j.jretai.2020.05.004.

[15] J. A. Sokolowski and C. M. Banks. *Modeling and Simulation Fundamentals: Theoretical Underpinnings and Practical Domains*. Hoboken, New Jersey: John Wiley & Sons Inc., 2010. ISBN: 978-0-470-48674-0.

[16] Alison Heppenstall, Nick Malleson, and Andrew Crooks. ""Space, the Final Frontier": How Good are Agent-Based Models at Simulating Individuals and Space in Cities?" In: *Systems* 4.1 (Mar. 2016). Number: 1 Publisher: Multidisciplinary Digital Publishing Institute, p. 9. ISSN: 2079-8954. DOI: 10.3390/systems4010009. URL: https://www.mdpi.com/2079-8954/4/1/9 (visited on 02/25/2025).

[17] E. Ozgormus and A. E. Smith. "A data-driven approach to grocery store block layout". In: *Computers & Industrial Engineering* 139 (2020), p. 105562. DOI: 10.1016/j.cie.2018.12.009. URL: https://www.sciencedirect.com/science/article/pii/S0360835218306144 (visited on 01/30/2025).

[18] R. R. Seva. "Grocery Product Arrangement Using Cluster Analysis". In: *Proceedings of the International MultiConference of Engineers and Computer Scientists*. Vol. 2239. 2019, pp. 549–463. URL: https://d1wqtxts1xzle7.cloudfront.net/76590667/IMECS2019_pp459-463-libre.pdf.

[19] M. A. Rothenberger K. Peffers T. Tuunanen and S. Chatterjee. "A Design Science Research Methodology for Information Systems Research". In: *Journal of Management Information Systems* 24.3 (2007), pp. 45–77. DOI: 10.2753/MIS0742-1222240302.

[20] Volker et al. Grimm. "The ODD protocol: A review and first update". In: *Ecological Modelling* 221.23 (Nov. 2010), pp. 2760–2768. DOI: 10.1016/j.ecolmodel.2010.08.019.

[21] U. Wilensky. *NetLogo*. Version 6.4.0. Publisher: Center for Connected Learning and Computer-Based Modeling, Northwestern University. Evanston, IL, 1999. URL: http://ccl.northwestern.edu/netlogo/hubnet.html.

[22] L. W. Turley and R. E. Milliman. "Atmospheric Effects on Shopping Behavior: A Review of the Experimental Evidence". In: *Journal of Business Research* 49 (2 Aug. 2000), pp. 193–211. DOI: 10.1016/S0148-2963(99)00010-7.

[23] J. Dorismond. "Supermarket optimization: Simulation modeling and analysis of a grocery store layout". In: *2016 Winter Simulation Conference (WSC)*. ISSN: 1558-4305. Dec. 2016, pp. 3656–3657. DOI: 10.1109/WSC.2016.7822385. URL: https://ieeexplore.ieee.org/document/7822385 (visited on 01/30/2025).

[24] K. Holmgren. "Customer Path Generation Simulation for Selection from Proposed Grocery Store Layouts". In: *2021 Winter Simulation Conference*. 2021, pp. 1–11. DOI: 10.1109/WSC52266.2021.9715297.

[25] S. Y. Lee. "Impacts of Store Crowding on Shopping Behavior and Store Image". In: *Journal of Asian Architecture and Building Engineering* 10.1 (2011), pp. 133–140. DOI: 10.3130/jaabe.10.133.

[26] A. Alawahdi and S. Yoon. "Shopping Behavioral Intentions Contributed by Store Layout and Perceived Crowding: An Exploratory Study Using Computer Walk-Through Simulation". In: *Journal of Interior Design* 41.4 (2016), pp. 29–46. ISSN: 10717641. DOI: 10.1111/joid.12077.

[27] C. Levenier M. Goić and R. Montoya. "Drivers of Customer Satisfaction in the Grocery Retail Industry: A Longitudinal Analysis Across Store Formats". In: *Journal of Retailing and Consumer Services* 60 (May 2021), p. 102505. DOI: 10.1016/j.jretconser.2021.102505.

[28] M. J. Bitner. "Servicescapes: The Impact of Physical Surroundings on Customers and Employees". In: *Journal of Marketing* 56.2 (Apr. 1992), pp. 57–71. DOI: 10.2307/1252042.

[29] *Microsoft Copilot*. private communication. Apr. 2025.

[30] U. Wilensky and W. Rand. *An Introduction to Agent-Based Modeling: Modeling Natural, Social, and Engineered Complex Systems with NetLogo*. MIT Press, 2015.

[31] S. Liu and C. Liu. "Study of QL Supermarket Layout Optimization Scheme Based on SLP Method and Logistics Data Analysis". In: *2022 IEEE 4th Eurasia Conference on IOT, Communication and Engineering (ECICE)*. Oct. 2022. DOI: 10.1109/ECICE55674.2022.10042860.

[32] A. I. Jiménez-Zarco M. P. Martínez-Ruiz and R. Cascio. "Assessing the maximum level of customer satisfaction in grocery stores". In: *International Journal of Retail & Distribution Management* 39.7 (2011), pp. 504–521. DOI: 10.1108/09590551111144897.

[33] A. Hernández-García A. S. Ananda H. Hanny and P. Prasetya. "'Stimuli Are All Around'—The Influence of Offline and Online Servicescapes in Customer Satisfaction and Repurchase Intention". In: *Journal of Theoretical and Applied Electronic Commerce Research* 18.1 (Mar. 2023), pp. 524–547. DOI: 10.3390/jtaer18010027. URL: https://www.mdpi.com/0718-1876/18/1/27.

[34] F. Daňo and D. Lesáková. "The Role of Environmental Stimuli in Shopping Evaluation and Responses 1". In: *Ekonomicky Casopis* 66.5 (2018), pp. 465–478. ISSN: 00133035.

[35] K. Bäckström and U. Johansson. "Creating and consuming experiences in retail store environments: Comparing retailer and consumer perspectives". In: *Journal of Retailing and Consumer Services* 13.6 (2006), pp. 417–430. DOI: 10.1016/j.jretconser.2006.02.005.

[36] L. Esbjerg et al. "An integrative conceptual framework for analyzing customer satisfaction with shopping trip experiences in grocery retailing". In: *Journal of Retailing and Consumer Services* 19.4 (July 2012), pp. 445–456. DOI: 10.1016/j.jretconser.2012.04.006.

[37] R. N. Mattick P. Huddleston J. Whipple and Lee S. J. "Customer satisfaction in food retailing: comparing specialty and conventional grocery stores". In: *International Journal of Retail & Distribution Management* 37.1 (Jan. 2009), pp. 63–80. DOI: 10.1108/09590550910927162.

[38] M. Al-Ali, N. E. N. Bazin, and S. M. Shamsuddin. "Key Determinants of Customer Satisfaction: Evidence from Malaysia Grocery Stores". In: *Journal of Theoretical and Applied Information Technology* 74.3 (Apr. 2013), pp. 286–299. URL: https://www.jatit.org/volumes/Vol74No3/2Vol74No3.pdf.

[39] D. H. Simon et al. "Employee attitudes, customer satisfaction, and sales performance: Assessing the linkages in US grocery stores". In: *Managerial and decision economics* 30.1 (Jan. 2009), pp. 27–41. DOI: 10.1002/mde.1433.

[40] E. W. M. I. Gómez, McLaughlin, and D. R. Wittink. "Customer Satisfaction and Retail Sales Performance: An Empirical Investigation". In: *Journal of Retailing* 80 (2004), pp. 265–278. DOI: 10.1016/j.jretai.2004.10.003.

[41] P. Johannesson and E. Perjons. *An Introduction to Design Science*. 2nd. Cham: Springer International Publishing, 2021. ISBN: 978-3-030-78132-3. DOI: 10.1007/978-3-030-78132-3.

[42] Jan Vom Brocke, Alan Hevner, and Alexander Maedche. "Introduction to Design Science Research". In: *Design Science Research. Cases*. Ed. by Jan Vom Brocke, Alan Hevner, and Alexander Maedche. Series Title: Progress in IS. Cham: Springer International Publishing, 2020, pp. 1–13. ISBN: 978-3-030-46781-4. DOI: 10.1007/978-3-030-46781-4_1. URL: http://link.springer.com/10.1007/978-3-030-46781-4_1 (visited on 02/26/2025).

[43] Alan R Hevner. "A Three Cycle View of Design Science Research". en. In: *Scandinavian Journal of Information Systems* (2007).

[44] B. J. Oates. *Researching Information Systems and Computing*. London: SAGE Publications, 2006. ISBN: 1-4129-0223-1.

[45] E. Orduna-Malea A. Martín-Martín M. Thelwall and E. Delgado López-Cózar. "Google Scholar, Microsoft Academic, Scopus, Dimensions, Web of Science, and OpenCitations' COCI: A Multidisciplinary Comparison of Coverage via Citations". In: *Scientometrics* 126 (2021), pp. 871–906. DOI: 10.1007/s11192-020-03690-4.

[46] K. Miwa and S. Takakuwa. "Simulation modeling and analysis for in-store merchandizing of retail stores with enhanced information technology". In: *2008 Winter Simulation Conference*. ISSN: 1558-4305. Dec. 2008, pp. 1702–1710. DOI: 10.1109/WSC.2008.4736256. URL: https://ieeexplore.ieee.org/document/4736256 (visited on 02/19/2025).

[47] J. M. Watson J. W. Gruber R. Smiddy and E. J. Williams. "Simulation helps local grocery store compete effectively against large chains". In: *2015 International Conference on Industrial Engineering and Operations Management (IEOM)*. Mar. 2015, pp. 1–4. DOI: 10.1109/IEOM.2015.7093695. URL: https://ieeexplore.ieee.org/document/7093695 (visited on 01/27/2025).

[48] V. Grimm et al. "Using the ODD protocol and NetLogo to replicate agent-based models". In: *Ecological Modelling* 501 (Feb. 2025), p. 110967. DOI: 10.1016/j.ecolmodel.2024.110967.

[49] I. Jacobson J. Rumbaugh and G. Booch. *Unified Modeling Language Reference Manual, The (2nd Edition)*. Pearson Higher Education, 2004. ISBN: 0321245628.

[50] E. Rodgers. *The State of Grocery Shopping*. Tech. rep. Drive Research, 2024.

[51] W. R. Darden B. J. Babin and M. Griffin. "Work and/or Fun: Measuring Hedonic and Utilitarian Shopping Value". In: *Journal of Consumer Research* 20.4 (Mar. 1994), pp. 644–657. DOI: 10.1086/209376.

[52] B. Kitchenham et al. “Systematic literature reviews in software engineering–a systematic literature review”. In: *Information and Software Technology* 51.1 (2009), pp. 7–15. DOI: 10.1016/j.infsof.2008.09.009.

[53] V. D. Tran. “Assessing the relationship between perceived crowding, excitement, stress, satisfaction, and impulse purchase at the retails in Vietnam”. In: *Cogent Business & Management* 7.1 (2020), p. 1858525. DOI: 10.1080/23311975.2020.1858525.

[54] V. Grimm et al. “The ODD Protocol for Describing Agent-Based and Other Simulation Models: A Second Update to Improve Clarity, Replication, and Structural Realism”. In: *Journal of Artificial Societies and Social Simulation* 23.2 (2020), p. 7. DOI: 10.18564/jasss.4259.

[55] K. A. Machleit, S. A. Eroglu, and S. P. Mantel. “Perceived Retail Crowding and Shopping Satisfaction: What Modifies This Relationship?” In: *Journal of Consumer Psychology* 9.1 (2000), pp. 29–42. DOI: https://doi.org/10.1207/s15327663jcp0901_3.

Appendix A

ODD Description of the Model

This section describes the constructed model. Sections 7.1 - 7.3 represent the O (Overview) and describe the model's overall purpose, its agents, states, and the model's scale. Section 7.4 represents the first D (Design concepts), discussing which of the 11 design concepts within ODD appear in this model and are used during the model development. Finally, sections 7.5 - 7.7 represent the second D (Details), which describe the model's initial conditions, environmental variables, and its submodels.

The protocol aims to describe the model in its entirety; it is a complete and detached representation of the model independent of the code or other parts of this text.

The protocol is sourced from and interpreted using the following references: [20, 54, 48].

A.1 Purpose and patterns

A.1.1 Purpose

The purpose of the model is to investigate and predict how a grocery store's department layout affects the store's revenue and its customers' experience of the visit. The model should be able to examine and compare different layouts to see how these affect customer satisfaction and revenue. The model can be used to support the planning of grocery store designs in the future and could be further developed and generalized to other types of stores going forward.

For the model to be considered to fulfill its purpose, it must be possible to demonstrate with the model that a grocery store's layout affects the store's revenue and its customers' satisfaction. A desirable outcome of the model would be to extend and deepen the understanding of how customer satisfaction and revenue affect each other and the relationships that exist between the two.

A.1.2 Patterns

The model is based on patterns found in real grocery stores. Primarily, customer behaviors are based on patterns from reality and previous research. Examples include:

- A customer more frequently buys products from certain departments in a grocery store [50].

- A customer enters a store with the intention to purchase something.
- A customer pays and then leaves the store.
- A customer queues at a checkout.
- A customer avoids other customers and dislikes congestion [13].

A.2 Entities, state variables and scales

A.2.1 Entities and state variables

This section lists the entities and agents present in the model and the variables they store. Their behaviors are described later in the protocol.

Customer - Agent		
State variable	*Type*	*Description*
shopping-list	List of items	What the customer wishes to purchase
shopping-cart	List of items	What the customer has purchased so far
items-not-found	List of items	Items the customer cannot find in the store ("out of stock")
satisfaction-o-meter	Integer	How satisfied the customer is
time-in-store	Integer	Number of ticks since the customer entered the store
in-line?	Boolean	True/false indicating whether the customer is standing in line at the checkout
time-since-entering-section	Integer	Time since the customer entered the current section they are in
headed-towards-POS?	Boolean	True/false indicating whether the customer is heading towards the checkout

Tabell A.1: Customer entity state variables

Customer: Customers are included as the only type of agent in this simulation because the focus of the study is based on customer behavior in the store. We have not included other agents such as store staff because the simulation results do not involve any parameters related to personnel. The purpose of the simulation is to investigate how a grocery store's layout affects its customers' satisfaction and its revenue; therefore, including staff would be relatively irrelevant and would add unnecessary complexity to the model.
Each customer has a shopping list and a shopping cart; these represent each customer's goal with their visit and how well the goal is fulfilled at a specific point in the simulation. A customer's satisfaction is a scale of how pleased the customer is with their visit; these values are used to evaluate the simulation and serve as a measure both of whether the model fulfills its purpose and how the specific layout affects its customers. Satisfaction is an arbitrary number intended for comparative purposes; various events during the simulation affect a customer's satisfaction—congestion and time spent shopping are examples of this.

Section		
State variable	*Type*	*Description*
section-product-type	String	The type of products belonging to the section
section-items	List of items	The items available in the section
shopping-list-percent	Integer	Probability that items from this section are found in a customer's shopping cart

Tabell A.2: Section entity state variables

Section: The grocery store consists of different sections where shelves with products are located. The simulation aims to investigate how the placement of the sections affects the factors of customer satisfaction and revenue, and for the simulation to be considered successful, there should be a correlation between these. The sections also practically have a size and position within the store.

Items: Items are a collective entity to link a customer's shopping list and a section. A customer has lists of items they wish to buy, items they have purchased, and items they failed to find. Each item is linked to a section based on the state variable *product-type*.

Item		
State variable	*Type*	*Description*
name	String	The name of the item
price	Decimal	The price of the item
product-type	String	The section the item belongs to

Tabell A.3: Item entity state variables

Checkout		
State variable	*Type*	*Description*
queue	List of customers	The queue at the checkout
service-time-counter	Integer	A countdown for how much time remains to serve the customer at the front of the queue

Tabell A.4: Checkout entity state variables

Checkout: A checkout is, like a section, a physical part of the grocery store but has different properties and functions than the sections. Previous research has shown that a grocery store's checkout layout significantly affects customer satisfaction and the store's revenue [10].

Doors		
State variable	*Type*	*Description*
is-entrance?	Boolean	Whether the door is an entrance or exit

Tabell A.5: Door entity state variables

Entrance/Exit: To represent the store's connection to the outside world, two agents of type *door* are used. These have a position within the store and a variable indicating whether they represent an entrance or an exit.

A.2.2 Scales

Each tick during the model's execution represents 1/4 minute, i.e., 15 seconds, and each unit of length represents 1 meter. This means that the total area of the model is $32m * 32m = 1024m^2$.

A.3 Process overview and scheduling

A.3.1 Initialization

To initialize the simulation, NetLogo runs the *setup* procedure. This is responsible for initializing all entities with correct values and preparing for the simulation. The *setup* procedure in turn calls several routines that initialize global variables, items, sections, and patches. The pseudocode below illustrates the relationship between the procedures invoked during the model's initialization. Conditional statements and logic have been abstracted away to clearly show the call hierarchy. See Section A.5 for more information on initialization.

```
to setup
    setup-globals-and-settings
    setup-items
    setup-sections
    setup-patches
end

to setup-sections
    ; method call "Section name" [probability of occurrence in shopping list]
    setup-one-section "Dairy" 82
    setup-one-section "Fruit and Vegetables" 80
    setup-one-section "Snacks" 76
    setup-one-section "Bread & Bakery" 71
    setup-one-section "Frozen" 70
    setup-one-section "Pantry" 65
    setup-one-section "Deli" 62
    setup-one-section "Household" 59
    setup-one-section "Beverages" 53
    setup-one-section "Hygiene" 50
    setup-one-section "Pet" 34
    setup-one-section "Children" 7

    remove-some-items
end

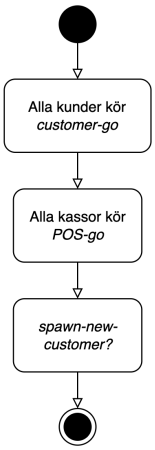
to remove-some-items
```

```
    remove-item-from-section this-item
end

to setup-patches
    setup-[layout-name]-layout
end

to setup-[layout-name]-layout ; layout setup procedures differ only in coordinate values
    for all sections
        setup-one-section-patch coordinates color
    setup-one-POS coordinates
    setup-one-POS coordinates
    setup-doors coordinates
end

to setup-one-section-patch [coordinates color]
    set-xy-variables coordinates
    set-xy-pos-from-variables
end
```



Figur A.1: Activity diagram: *go*

A.3.2 Execution

Once the simulation starts, the model automatically executes the *go* procedure every time unit (henceforth called a *tick*). From *go*, all customer agents are instructed to perform the *customer-go* procedure in random order, and all checkouts execute the *POS-go* procedure. The *go* procedure concludes by running the *spawn-new-customer?* procedure. Figure A.1 shows an activity diagram of the *go* procedure. The subroutines called by *go* are described below using text, diagrams, and

pseudocode based on the NetLogo code.

customer-go

This procedure is executed every *tick* for all *customer* agents in the store via the *ask* command, which in NetLogo means agents are asked to run a procedure in random order. Figure A.2 shows all procedure calls made from within *customer-go*. The indentation depth represents the call origin. In pseudocode:

```
to customer-go
    spawn-new-item?
    move
    pick-up-item-attempt
    get-in-line-attempt
    exit-store-attempt
end

to move
    set-target
end

to set-target
    check-for-items-to-shop
    detect-closeby-customers
end

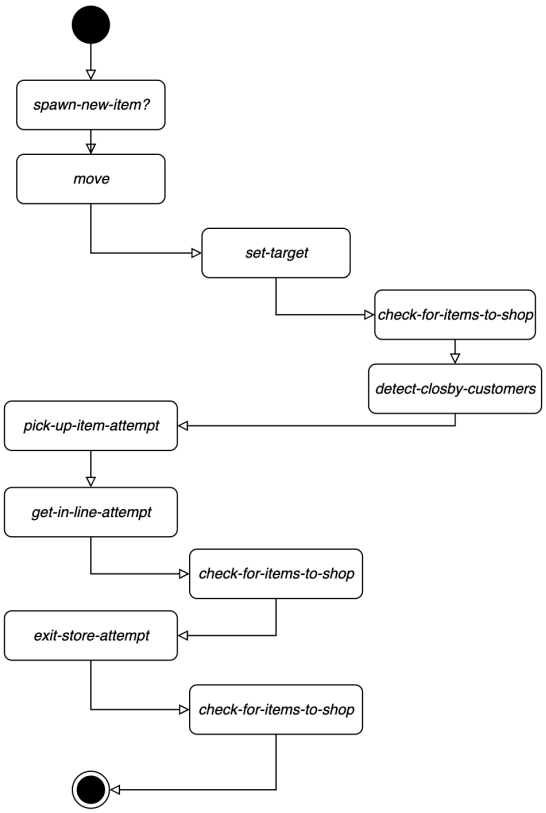
to get-in-line-attempt
    check-for-items-to-shop
end

to exit-store-attempt
    check-for-items-to-shop
end
```

POS-go

All checkouts in the system execute this procedure every *tick* via the *ask* command, meaning the order in which checkouts run the procedure is random. Figure A.3 shows the procedures called from *POS-go*. Below is the pseudocode version of the procedure. See Section A.7.8 for clarification.

```
to POS-go
    customer-check-out
    queue-forward-step
end
```



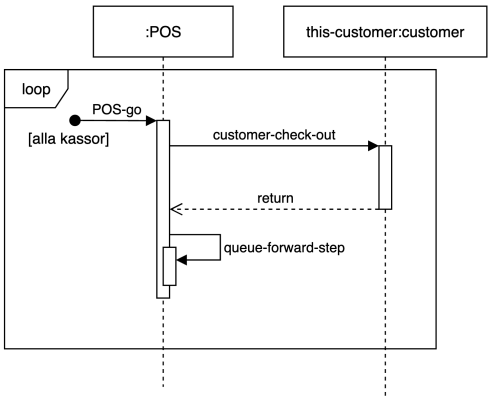
Figur A.2: Procedure calls from *customer-go*

spawn-new-customer?

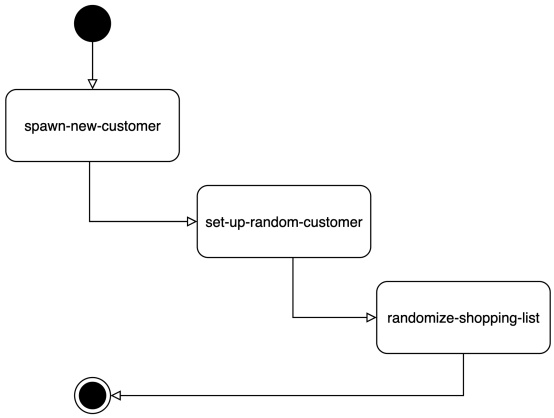
This procedure is responsible for introducing new customers to the store by creating a new customer using the *setup-random-customer* procedure. Figure A.4 illustrates the procedure calls.

```
to spawn-new-customer?
    if (will-spawn?)
        setup-random-customer
end

to setup-random-customer
    randomize-shopping-list
end
```



Figur A.3: Procedure calls from *POS-go*



Figur A.4: Procedure calls from *spawn-new-customer?*

A.4 Design concepts

The design concepts below are described in detail in [20, 54, 48]. Here, we explain how these concepts relate to the model under study.

A.4.1 Basic principles

The model is designed based on principles from previous research on grocery stores, their layout, customer satisfaction, and how these factors affect store revenue. Customers' shopping lists are generated according to studies on the likelihood of customers purchasing items from different store sections; this is implemented and described in detail in A.7.10 *randomize-shopping-list* [50]. Customer behavior in the store is grounded in research indicating that customers dislike congestion, which influences their movement patterns and how crowding impacts customer satisfaction (see A.7.3 *detect-closeby-customers*) [55, 13]. Another foundational principle is that customer satisfac-

tion depends on how well their expectations for the shopping occasion are met, reflected in the calculation of satisfaction related to time spent in-store (see A.7.4 *exit-store-attempt*) and in the procedure A.7.7 *pick-up-item-attempt*, which manages customer behavior when they cannot find desired items [36]. Finally, the idea that the probability of unplanned (impulse) purchases increases if customers are satisfied is used in A.7.18 *spawn-new-item* [53].

A.4.2 Emergence

Queue formations and congestion can arise as emergent phenomena from the behavior of agents.

A.4.3 Adaptation

Customers adapt to certain conditions. Primarily, they follow their shopping lists by purchasing the items included therein. Additionally, customer satisfaction influences behavior by increasing or decreasing the likelihood of impulse purchases. Customers also adapt based on their mood and location within the store, which affects where impulse purchases are made; see A.7.18 *spawn-new-item* for this process.

A.4.4 Objectives

The agents representing shopping customers aim to complete their shopping task — that is, to buy all (or as many as possible) of the items on their list, pay, and leave the store. Successful completion results in higher self-rated satisfaction, while failure lowers satisfaction.

A.4.5 Learning

The model does not include learning mechanisms.

A.4.6 Prediction

The agents do not perform predictions.

A.4.7 Sensing

Customer agents seek to avoid congestion by scanning their nearby environment for other customers (see A.7.3 for implementation details). They also sense which section the patch they occupy belongs to, influencing their purchasing opportunities.

A.4.8 Interaction

Customers queue at the checkout in the final stage of shopping, interacting with the checkout by paying for their goods before exiting the store. Agents also interact with other customers in the queue by positioning themselves behind the last customer in line.

A.4.9 Stochasticity

Some randomness exists in the model. New customers are generated randomly, and their shopping lists are randomized based on statistics from [50]. Certain items are randomly removed to simulate stockouts or the inability of customer agents to find items. See A.3.2, A.7.10, and A.7.12 for implementation details.

A.4.10 Collectives

Collectives are not included in the model.

A.4.11 Observation

Observed data include numerical values of store sales revenue and aggregate customer satisfaction for customers who shopped during simulation runs. Additionally, graphs show the number of customers present in each store section over time. Finally, the paths customers travel through the store during shopping are also observed.

A.5 Initialization

The model is initialized via the *setup* procedure, which is executed when the user clicks the setup button in the graphical user interface. This occurs before the simulation starts and ensures that all entities required at the start are created and assigned appropriate values.

Many of the initialization instructions are contained within five procedures named according to the formula *setup-"name"-layout*. The user can program their own layout in *setup-custom-layout*, while the existing layouts are largely similar. The description below is based on these existing layouts, with any differences between layouts explicitly noted.

Initialization includes only one random component, allowing the model to be initialized in nearly the same way multiple times if the user desires. The value of the *layout* variable determines which layout the model initializes to; thus, if the user selects the same layout in the GUI's *layout-chooser*, the model's initial state will be nearly identical, except for which items are missing from the store, which is determined randomly. See A.7.12 for details on the process of removing certain items.

The entities created during initialization are *sections* (departments), *items* (products), *POS* (points of sale or checkouts), and *doors* (entrances and exits). Additionally, patches are initialized.

A.5.1 Items

All items are initialized with three attributes: *name*, *price*, and *product-type*. They are assigned *size* 0 since they do not occupy physical space directly in the model but are linked to a department or list which in turn has a location. A total of 15 items per department (12 departments) are initialized, resulting in 180 items. All items are hardcoded with names, prices, and product types corresponding to their respective departments. Example code to create the item *Dishwashing Liquid*, costing 22 monetary units and belonging to the *Household* department:

```
create-items 1 [
    set size 0
    set name "Dishwashing Liquid"
    set price 22
    set product-type "Household"
]
```

A.5.2 Sections

Each layout contains the same 12 sections. These are initialized with values for *section-product-type* (i.e., department name), *section-items*, and *shopping-list-percent*, as well as *size* 0 since the physical location of the section is represented by the patches it is linked to (see A.5.5). Sections also receive a position defined by coordinates corresponding to the centroid of their linked patches. This positioning varies between layouts. Example initialization of the dairy section in layout 1 (actual NetLogo code involves several procedures but assigns the same values as below). The sections initialized are: *dairy, produce, snacks, bread & bakery, frozen, pantry, deli, household, beverages, hygiene, pets*, and *children*.

```
create-sections 1 [
    set size 0
    set section-product-type "Dairy"
    set section-items items with [product-type = "Dairy"]
    set shopping-list-percent 82
    setxy 0 -11
]
```

After all sections have been assigned their items, the *remove-some-items* procedure (see A.7.12) is run to randomly remove certain items from some sections. This procedure uses NetLogo's normal distribution random function. The mean number of missing items, *average-nbr-of-missing-items*, is initialized to 5. This process ensures variation between runs even if the same layout is used.

The state variable *shopping-list-percent* represents the probability that an item from the section is randomly included in a customer's shopping list. This percentage is used in procedure A.7.10 *randomize-shopping-list* and is based on the study *The State of Grocery Shopping* [50].

A.5.3 POS (Points of Sale)

In the existing layouts, two checkouts are initialized with *shape*, *size*, *color*, a *queue*, a *service-time-counter*, and a position. The two checkouts are identical at simulation start except for their coordinates, which differ between layouts. Example code to initialize one checkout:

```
create-POS 1 [
    set shape "square"
    set size 2
    set color red
    set queue []
    set service-time-counter 0
```

```
    setxy -6 15 ; for one checkout in layouts 1 and 2
]
```

A.5.4 Doors

All layouts include one entrance and one exit, initialized with values for *shape*, *size*, *color*, *is-entrance?*, and position. Doors are located along the southern edge of the simulation area, with y-coordinate -16. Example code to initialize an exit door:

```
create-doors 1 [
    set is-entrance? false
    setxy -1 -16
    set size 1
    set shape "square"
    set color 15.5
]
```

A.5.5 Patches

In the simulation, patches are linked to departments; if a customer occupies a patch linked to a department, they can purchase items belonging to that department. All patches are initialized with a department reference and a color associated with the department. Patches without a department are assigned *patch-section* value 0 and colored white. This process varies completely between layouts but, generally, each existing layout has at least one patch linked to each of the 12 departments, and all patches representing the same department are orthogonally connected. Example of initializing patches linked to the dairy department in layout 1:

```
let dairy-section one-of sections with [section-product-type = "Dairy"]
ask patches with
[pxcor >= -5 and pxcor <= 5 and pycor >= -14 and pycor <= 8] [
    set patch-section dairy-section
    set pcolor blue
]
```

A.6 Input data

The model does not process any input data.

A.7 Submodels

This section describes the routines in the model's code that have not yet been covered in section A.3 Process overview and scheduling or A.5 Initialization.

A.7.1 check-for-items-to-shop

This procedure aims to examine what items a customer still has left on their shopping list. It uses the global variable *return-item* as a return value, which is assigned either an item the customer still needs to purchase or the keyword *nobody* if the customer has nothing left to buy. In detail, the routine works as follows: it is called by a customer and thus has access to the customer's shopping list, which is iterated through until an item is found that is neither in the customer's shopping cart (*shopping-carts*) nor on the list of items the customer has not yet found (*items-not-found*). If such an item is found, it is returned.

The procedure is called from A.7.13 *set-target*, A.7.5 *get-in-line-attempt*, and A.7.4 *exit-store-attempt*.

A.7.2 customer-check-out

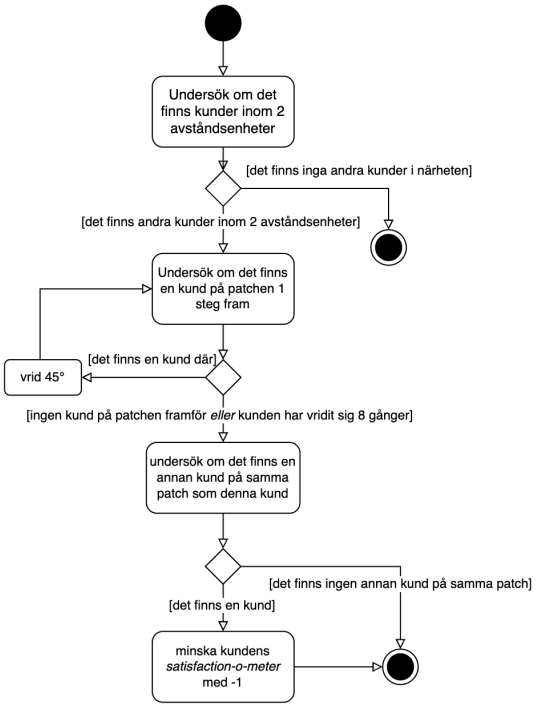
This procedure is responsible for allowing a customer to pay at a checkout. This is done by iterating over all items in the customer's shopping cart (*shopping-cart*) and summing their prices. The global variable *total-revenue* is then increased by the customer's purchase. The procedure is called from A.7.8 *POS-go*.

A.7.3 detect-closeby-customers

This procedure aims to identify nearby customers and, if possible, avoid them during a customer's movement process. The procedure works as follows: using NetLogo's *in-radius* function, it checks whether there are any other customers within 2 distance units. If so, it checks whether there is a customer on the patch directly in front of the customer who called the procedure. If so, the customer turns 45° either to the right or left (randomly determined using NetLogo's *random* function). This is repeated until a patch without any customers is found or up to a maximum of 8 times. Finally, it checks whether there are any other customers on the same patch as the customer executing the procedure. If so, the customer's satisfaction (*satisfaction-o-meter*) is decreased by 1 step. This process is illustrated in an activity diagram in Figure A.5.
This procedure is based on the idea that customers may become dissatisfied due to crowding during a shopping trip [55, 13], and therefore both try to avoid other customers in the store and have their satisfaction meter decrease when standing on the same patch as another customer. The procedure is unit-tested, and through these tests, the value of 45° was chosen as it resulted in a reasonable movement pattern. The procedure is called from A.7.13 *set-target*.

A.7.4 exit-store-attempt

This procedure is responsible for checking whether it is possible for a customer to exit the store based on a number of criteria. If it is possible, the process required to let a customer exit the store and "die" in the simulation is also executed. The criteria for allowing the customer to exit are that the customer must have finished shopping, which is checked using the global variable *return-item* returned from the procedure A.7.1 *check-for-items-to-shop*. The customer must also be located at an exit (i.e., a *door* entity with *is-entrance?* set to *false*) and must have paid, which is checked



Figur A.5: Activity diagram: detect-closeby-customers

using the customer's state variables *in-line?*, *headed-towords-POS?*, and *has-payed?*.

If the customer is to exit the store, their final satisfaction is calculated by comparing how long they expected to be in the store with how long they actually spent there. This is based on a theory within the utilitarian shopping paradigm, which suggests that a customer is satisfied if they get what they expected from their shopping experience [36]. The calculation is as follows, where t is the time since the customer entered the store, i is the number of items on the customer's shopping list (*shopping-list*), and k is a constant retrieved from the global variable *expected-time-per-item*, which the user can adjust via a slider in the graphical interface.

$$f(t, i) = \frac{ki - t}{i} \tag{A.1}$$

Once this is calculated, the customer's state variable *satisfaction-o-meter* is adjusted accordingly, and the global variable *total-satisfaction* is increased by the value of the customer's *satisfaction-o-meter*. Finally, the NetLogo imperative *die* is used to remove the customer from the simulation, thereby simulating their exit from the store.

The procedure is called from A.3.2 *customer-go*.

A.7.5 get-in-line-attempt

This procedure ensures that a customer queues up if they have finished shopping but have not yet queued or paid. This is done by first checking whether the customer has finished shopping using the global variable *return-item*, which is returned from the procedure A.7.1 *check-for-items-to-shop*, and then checking whether the customer is at a checkout or within 2 distance units of a customer who is in line at the checkout the customer is closest to. If so, the customer joins the queue by having the checkout entity add the customer to the end of its *queue* state variable and by setting their own variables *in-line?* and *in-line-to* to *true* and the nearest checkout entity, respectively.

The procedure is called from A.3.2 *customer-go*.

A.7.6 move

This procedure is responsible for moving the customer who executes the procedure forward. It begins by calling the procedure A.7.13 *set-target*, then moves the customer forward by 0.25 distance units, provided the customer is not in line or heading toward a checkout, which is checked using the customer's state variables *in-line?* and *headed-towords-POS?*. Then, the customer's state variables *time-in-store* and *time-since-entering-section* are incremented by 1, provided the customer is on a patch linked to a department. If not, *time-since-entering-section* is instead set to 0.

The procedure is called from A.3.2 *customer-go*.

A.7.7 pick-up-item-attempt

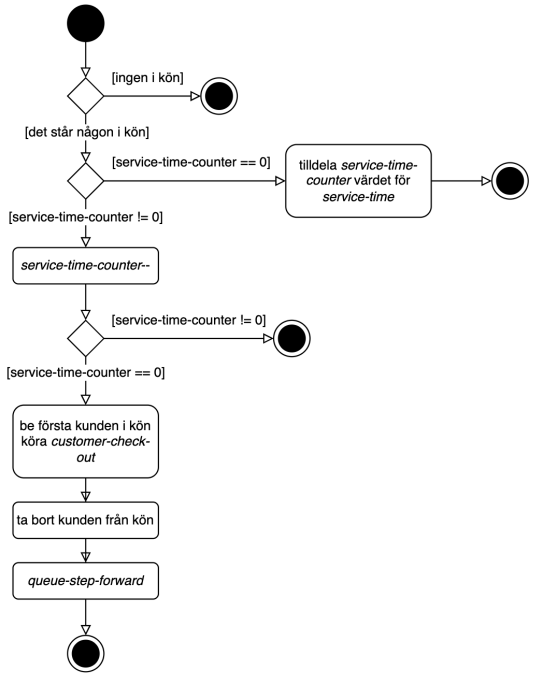
This procedure enables a customer to pick up an item from the department they are currently in. This is done by first checking that the customer is on a patch linked to a department via the patch's state variable *patch-section*. Then, the customer's *shopping-list* is iterated through, and if the item being examined has not yet been purchased (i.e., is not in the customer's *shopping-cart*) and fits in the department the customer is in (the item is found in the department's *section-items*), it is added to the customer's shopping cart (*shopping-cart*). If the purchase is successful, the customer's *satisfaction-o-meter* is also incremented by 1.

During the development of this procedure, it was decided to iterate through the customer's shopping list rather than using another method, as the procedure is executed by the customer and their state variables are therefore easily accessible. The decision to increment the customer's satisfaction at this point is motivated by the idea that customers who get what they expect from their shopping experience become satisfied [36]. The procedure is called from A.3.2 *customer-go*.

A.7.8 POS-go

This procedure represents a time step, *tick*, for a checkout in the simulation. It uses the checkout's state variable *service-time-counter* to check whether the customer at the front of the queue (state variable *queue*) has been served long enough. If the customer is finished at the checkout, NetLogo's *ask* imperative is used to instruct the customer to execute procedure A.7.2 *customer-check-out*. The customer is also removed from the queue, and their state variables *headed-towords-POS?* and *has-payed?* are updated to *false* and *true*, respectively. Finally, procedure A.7.9 *queue-forward-step*

is called. The global variable *service-time* is used to reset the counter for the next customer in line and is initialized to 10 during *set-up* to simulate realistic queues.



Figur A.6: Activity diagram: *POS-go*

Figure A.6 shows an activity diagram of the procedure. The procedure is called from the *go* procedure; see section A.3.2 Execution for more information.

A.7.9 queue-forward-step

This procedure ensures that all customers in the queue of the checkout entity executing the procedure take a step forward toward the checkout. This is done using NetLogo's *ask* imperative, which instructs all customers in the checkout's *queue* state variable to turn toward the checkout and move forward by 0.25 distance units. If there is only one customer in the queue, they move all the way to the checkout regardless of the distance. The procedure is called from procedure A.7.8 *POS-go*.

A.7.10 randomize-shopping-list

This procedure aims to create a shopping list for a customer. It does so by iterating through all departments and generating a random number between 0 and 100. This number is compared with the department's state variable *shopping-list-percent*, which is based on a study of which departments customers typically purchase items from [50]. If the random number is lower than *shopping-list-percent*, a new random number is generated between 1 and the number of items in

the department. That number of items is then selected from the department, and if they are not already on the shopping list, they are added to the customer's shopping list. The procedure is called from procedure A.7.14 *set-up-random-customer*.

A.7.11 remove-item-from-section

This procedure ensures that the item received as an argument is removed from the department it belongs to. This is done by retrieving the department using the item's state variable *product-type* and then reassigning the department's *section-items* state variable without the item to be removed. A string representation of the item is then added to the global variable *items-removed*, which is a list of all removed items. The procedure is called from A.7.12 *remove-some-items*.

A.7.12 remove-some-items

This procedure is responsible for removing an arbitrary number of items from the simulation to simulate that the customer cannot find what they are looking for. The procedure uses NetLogo's *random-normal* function to generate a randomly normally distributed value. The global variable *average-nbr-of-missing-items*, initialized to 5, is used along with a standard deviation of 2. If the value is less than 0, no items are removed. Then, the corresponding number of items is instructed to execute procedure A.7.11 *remove-item-from-section* and are thus removed from their department.

The procedure is called during the initialization of departments from procedure *set-up-sections*; see A.5.2 Sections.

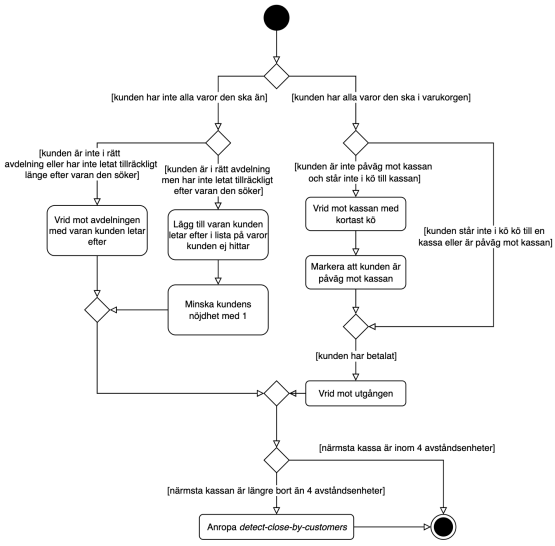
A.7.13 set-target

This procedure is executed by customers during the simulation to modify their direction. If the customer has not yet collected all items on their shopping list, it checks whether the customer is in the department associated with the item they are looking for. If so, the customer's state variable *time-since-entering-section* is compared with the global variable *max-time-spent-searching*, which is initialized to 10 time units. If *time-since-entering-section* is less than *max-time-spent-searching*, or if the customer is not in the department where the item is located, the customer turns toward the correct department. Otherwise, the item is added to the customer's *items-not-found* state variable, and *satisfaction-o-meter* is decremented by 1. If the customer has already collected all desired items, it checks whether the customer is heading toward a checkout or is already in line. If the answer to both is no, the customer turns toward the checkout with the shortest queue in the store, and their state variable *headed-towords-POS?* is set to *false*. Otherwise, it checks whether the customer has already paid, and if so, the customer turns toward an exit. This process is illustrated in an activity diagram in Figure A.7.

The procedure is called from the customer's movement procedure A.7.6 *move*.

A.7.14 set-up-random-customer

This procedure creates a new customer using NetLogo's *create-* function. The customer's position is set to the store's entrance, i.e., the door entity with the state variable *is-entrance?* set to *true*. All customers receive the same values for the state variables listed in the table below upon initialization.



Figur A.7: Activity diagram: *set-target*

What differentiates them is their *shopping-list*. Procedure A.7.10 *randomize-shopping-list* is used to generate the customer's shopping list. The procedure is called from A.3.2 *spawn-new-customer*.

State Variable	**Value**
coordinates	entrance coordinates
shape	"person"
size	2
color	black
shopping-cart	[]
items-not-found	[]
satisfaction-o-meter	0
time-in-store	0
in-line?	false
headed-towords-POS?	false
has-payed?	false

Tabell A.6: Initialization of a customer

The table describes the initial values for a customer's state variables. The value 2 for *size* corresponds to 2 distance units in the model, and [] represents an empty list.

A.7.15 set-xy-pos-from-variables

This procedure sets the caller's position based on the current values of the global variables *xmin, xmax, ymin*, and *ymax*. This is done using the formula below for both the x and y coordinates:

$$cor = min + \frac{max - min}{2}$$

The procedure is called from the *setup-one-section-patch* procedure (see section A.5.2 Sections).

A.7.16 set-xy-variables

This method receives four parameters, which are then assigned to four global variables. It is called from the *setup-one-section-patch* procedure (see section A.5.2 Sections).

Parameter	this-xmin	this-xmax	this-ymin	this-ymax
Global variable	xmin	xmax	ymin	ymax

Tabell A.7: Input and output for *set-xy-variables*

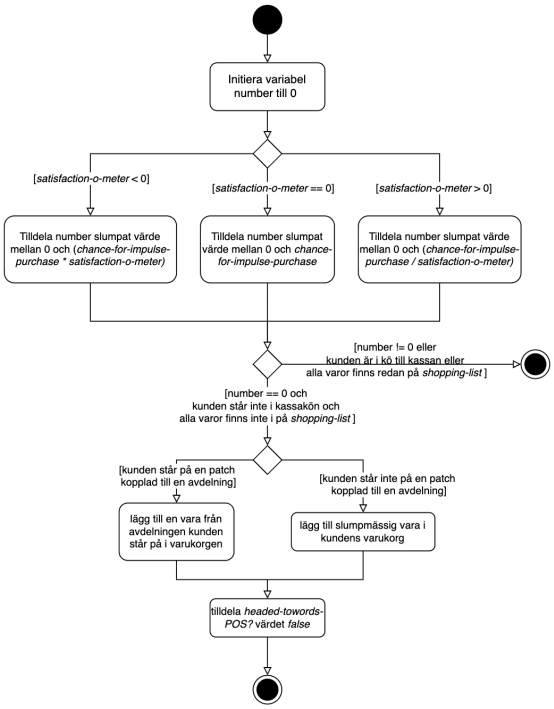
A.7.17 spawn-new-customer

This procedure is responsible for introducing new customers into the store. It begins by checking whether the number of customers is lower than the global variable *max-num-of-customers*, which the user can set via a slider in the graphical interface. Then, a number is randomly generated between 0 and the value of the global variable *chance-for-new-customer*, which can also be adjusted by the user. If the generated number is 0, procedure A.7.14 *set-up-random-customer* is called. The procedure is called from the *go* procedure; see sections A.3.2 Execution and A.3.2 spawn-new-customer for more information.

A.7.18 spawn-new-item?

This procedure is called from A.3.2 *customer-go* and may result in an impulse purchase by a customer. The procedure begins by generating a random number, and if that number is 0, an item is added to the customer's shopping cart. Below is how the number is generated based on the value of *satisfaction-o-meter* (satisfaction). The global variable *chance-for-impulse-purchase* (impulse) is used in the calculation and is set by the user via a slider in the graphical interface. The calculation is as follows, based on the idea that a satisfied customer is more likely to make an impulse purchase [53].

$$\begin{aligned} satisfaction < 0 &\Rightarrow number = random(impulse * satisfaction) \\ satisfaction = 0 &\Rightarrow number = random(impulse) \\ satisfaction > 0 &\Rightarrow number = random(\frac{impulse}{satisfaction}) \end{aligned} \tag{A.2}$$



Figur A.8: Activity diagram: *spawn-new-item?*

If the generated number is 0, and the customer is not in line at a checkout, and the customer does not already have all possible items on their shopping list, an item is added to the shopping list. If the customer is on a patch linked to a department, an item from that department is selected using NetLogo's *one-of* function and added to the shopping list if it is not already there. Otherwise, a random item is selected and added regardless of which department it belongs to. Figure A.8 shows an activity diagram describing the process. The procedure is called from A.3.2 *customer-go*.

A.7.19 Print Procedures

The code also includes a number of print procedures used during development and available to the user during execution. These instruct the agent or agents in question to print the values of their state variables and correspond to the information NetLogo displays when the user selects `inspect agent ID`. Therefore, they are not described here. The code uses the *ask* imperative and the *print* function available in NetLogo.